



# Mobile Platform (Android) PlayCtrl Library SDK

Developer Guide

© 2020 Hangzhou Hikvision Digital Technology Co., Ltd. All rights reserved.

This Document (hereinafter referred to be “the Document”) is the property of Hangzhou Hikvision Digital Technology Co., Ltd. or its affiliates (hereinafter referred to as “Hikvision”), and it cannot be reproduced, changed, translated, or distributed, partially or wholly, by any means, without the prior written permission of Hikvision. Unless otherwise expressly stated herein, Hikvision does not make any warranties, guarantees or representations, express or implied, regarding to the Document, any information contained herein.

#### **LEGAL DISCLAIMER**

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE DOCUMENT IS PROVIDED "AS IS" AND "WITH ALL FAULTS AND ERRORS". HIKVISION MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT. IN NO EVENT WILL HIKVISION BE LIABLE FOR ANY SPECIAL, CONSEQUENTIAL, INCIDENTAL, OR INDIRECT DAMAGES, INCLUDING, AMONG OTHERS, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION OR LOSS OF DATA, CORRUPTION OF SYSTEMS, OR LOSS OF DOCUMENTATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, IN CONNECTION WITH THE USE OF THE DOCUMENT, EVEN IF HIKVISION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR LOSS.

# Contents

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| Chapter 1 Overview .....                                                  | 1  |
| 1.1 Introduction .....                                                    | 1  |
| 1.2 Product Scopes .....                                                  | 1  |
| 1.3 Conventions .....                                                     | 1  |
| Chapter 2 Update History .....                                            | 1  |
| 2.1 Version 7.4.0.X (2020.09) .....                                       | 1  |
| 2.2 Version 7.3.9.X (2020.01) .....                                       | 1  |
| 2.3 Version 7.3.8.X (2019.07) .....                                       | 2  |
| 2.4 Version 7.3.7.X (2018.12) .....                                       | 2  |
| 2.5 Version 7.3.5.X (2018.04) .....                                       | 3  |
| 2.6 Version 7.3.5.X (2017.11) .....                                       | 3  |
| 2.7 Version 7.3.4.X (2017.03) .....                                       | 3  |
| 2.8 Version 7.3.3.X (2016.10) .....                                       | 3  |
| Chapter 3 Error Code .....                                                | 5  |
| 3.1 Common Error Code .....                                               | 5  |
| 3.2 Fisheye Error Code .....                                              | 6  |
| Chapter 4 API Call Flow .....                                             | 7  |
| 4.1 Play in File Mode .....                                               | 7  |
| 4.2 Play in Stream Mode .....                                             | 8  |
| 4.3 Synchronous Playback .....                                            | 9  |
| 4.4 Fisheye .....                                                         | 10 |
| 4.5 Hardware Decoding .....                                               | 11 |
| 4.6 Pre-recording .....                                                   | 12 |
| 4.7 Display Private Information .....                                     | 13 |
| Chapter 5 API .....                                                       | 14 |
| 5.1 System Operation and Error Code .....                                 | 14 |
| 5.1.1 Get SDK Version No. and Build No. getSdkVersion .....               | 14 |
| 5.1.2 Get Error Code getLastError .....                                   | 14 |
| 5.1.3 Get Port Number getPort .....                                       | 14 |
| 5.1.4 Release Playing Port freePort .....                                 | 15 |
| 5.2 File Operation .....                                                  | 16 |
| 5.2.1 Open File openFile .....                                            | 16 |
| 5.2.2 Close File closeFile .....                                          | 16 |
| 5.2.1 Set File Index Callback setFileRefCB .....                          | 17 |
| 5.3 Stream Operation .....                                                | 18 |
| 5.3.1 Set Stream Input Mode setStreamOpenMode .....                       | 18 |
| 5.3.2 Open Stream File openStream .....                                   | 18 |
| 5.3.3 Close Stream File closeStream .....                                 | 19 |
| 5.3.4 Input Stream Data inputData .....                                   | 19 |
| 5.3.5 Set Encrypted Stream Callback Function SetEncryptTypeCallBack ..... | 20 |
| 5.4 Playing and Control .....                                             | 21 |
| 5.4.1 Start Playing play .....                                            | 21 |

|        |                                                                  |    |
|--------|------------------------------------------------------------------|----|
| 5.4.2  | Start Playing playEx .....                                       | 21 |
| 5.4.3  | Stop Playing stop .....                                          | 21 |
| 5.4.4  | Pause Playing pause .....                                        | 22 |
| 5.4.5  | Fast Forward fast .....                                          | 22 |
| 5.4.6  | Slow Forward slow .....                                          | 22 |
| 5.4.7  | Play Audio in Exclusive Mode playSound .....                     | 23 |
| 5.4.8  | Stop Playing Audio in Exclusive Mode stopSound .....             | 23 |
| 5.4.9  | Play Audio in Sharing Mode playSoundShare .....                  | 23 |
| 5.4.10 | Stop Playing Audio in Sharing Mode stopSoundShare .....          | 24 |
| 5.4.11 | Adjust Volume adjustWaveAudio .....                              | 24 |
| 5.4.12 | Set Playing Progress (Percentage) setPlayPos .....               | 25 |
| 5.4.13 | Get Playing Progress (Percentage) getPlayPos .....               | 25 |
| 5.4.14 | Set Played Time (ms) setPlayedTimeEx .....                       | 25 |
| 5.4.15 | Get Played Time (ms) getPlayedTimeEx .....                       | 26 |
| 5.4.16 | Set Playing Progress (Frame) setCurrentFrameNum .....            | 26 |
| 5.4.17 | Get Playing Progress (Frame) getCurrentFrameNum .....            | 26 |
| 5.4.18 | Set Synchronization of Audio and Video Streams syncToAudio ..... | 27 |
| 5.4.19 | Set Window Transparency setWindowTransparency .....              | 27 |
| 5.4.20 | Refresh Current Display Window refreshPlay .....                 | 27 |
| 5.4.21 | Get Online Positioning Offset getPlayTimeOffset .....            | 28 |
| 5.4.22 | Set Playing Mode SetAudioTrackParam .....                        | 29 |
| 5.4.23 | Set Absolute Timestamp for Video Playing SetAbsTimeFlag .....    | 29 |
| 5.4.24 | Set AGC Parameters SetAGCParam .....                             | 29 |
| 5.4.25 | Set Chunk Size for RTMP Stream SetDemuxParam .....               | 30 |
| 5.4.26 | Set ANR Algorithm Parameters setANRParam .....                   | 30 |
| 5.5    | Get Playing or Decoding Information .....                        | 31 |
| 5.5.1  | Get File Time Duration getFileTime .....                         | 31 |
| 5.5.2  | Get System Time getSystemTime .....                              | 31 |
| 5.5.3  | Get File Frame Number getFileTotalFrames .....                   | 31 |
| 5.5.4  | Get Current Frame Rate getCurrentFrameRate .....                 | 32 |
| 5.5.5  | Get Decoded Frame Number getPlayedFrames .....                   | 32 |
| 5.5.6  | Get Original Image Size getPictureSize .....                     | 32 |
| 5.5.7  | Set Expected Frame Rate setExpectedFrameRate .....               | 33 |
| 5.6    | Decoding and Control .....                                       | 34 |
| 5.6.1  | Set Video Frame Decoding Type setDecodeFrameType .....           | 34 |
| 5.6.2  | Set Skipping I Frame Decoding setIFrameDecInterval .....         | 34 |
| 5.6.3  | Decoding Callback setDecodeCB .....                              | 35 |
| 5.6.4  | Set Decoding Error Callback setDecodeErrCB .....                 | 36 |
| 5.6.5  | Set Decoding Callback (with Global Time) setDecodeCBEx .....     | 37 |
| 5.6.6  | Set File End CallBack setFileEndCB .....                         | 38 |
| 5.6.7  | Set Decoding Key setSecretKey .....                              | 39 |
| 5.6.8  | Set Decoding Tolerance Level setDecodeERC .....                  | 39 |
| 5.6.9  | Set Number of Decoding Threads setDecodeThreadNumber .....       | 40 |
| 5.7    | Display Operation .....                                          | 41 |

|        |                                                                                                          |    |
|--------|----------------------------------------------------------------------------------------------------------|----|
| 5.7.1  | Set or Add Display Region setDisplayRegion .....                                                         | 41 |
| 5.7.2  | Set or Add Display Region setDisplayRegionEx .....                                                       | 42 |
| 5.7.3  | Set or Destroy Display Window setVideoWindow .....                                                       | 44 |
| 5.7.4  | Set or Destroy Display Window setVideoWindowEx .....                                                     | 44 |
| 5.7.5  | Set Synchronous Playback Group setSyncGroup .....                                                        | 45 |
| 5.7.6  | Enable Feature Tracking Effect (for EZVIZ Client) enableSuperEyeEffect .....                             | 45 |
| 5.7.7  | Disable Feature Tracking Effect (for EZVIZ Client) disableSuperEyeEffect .....                           | 46 |
| 5.7.8  | Get Parameters of the Current Display Area getCurrentRegionRect .....                                    | 46 |
| 5.7.9  | Set Image Post-processing Parameters setImagePostProcessParameter .....                                  | 47 |
| 5.8    | Source Buffer Operation before Decoding .....                                                            | 47 |
| 5.8.1  | Get Remaining Data Size of Source Buffer getSourceBufferRemain .....                                     | 47 |
| 5.8.2  | Clear All Buffers resetSourceBuf .....                                                                   | 48 |
| 5.8.3  | Clear Specified Buffer resetBuffer .....                                                                 | 48 |
| 5.9    | Playing Buffer Operation after Decoding .....                                                            | 49 |
| 5.9.1  | Set Frame Number in Playing Buffer setDisplayBuf .....                                                   | 49 |
| 5.9.2  | Get Frame Number in Playing Buffer getDisplayBuf .....                                                   | 49 |
| 5.10   | Source Buffer and Playing Buffer Operation .....                                                         | 50 |
| 5.10.1 | Get Specified Buffer Size getBufferValue .....                                                           | 50 |
| 5.11   | Capture .....                                                                                            | 51 |
| 5.11.1 | Set Capture Callback Function setDisplayCB .....                                                         | 51 |
| 5.11.2 | Capture Callback (with Global Time) setDisplayCBEx .....                                                 | 52 |
| 5.11.3 | Capture BMP Picture getBMP .....                                                                         | 53 |
| 5.11.4 | Capture BMP Picture with Private Information getBmpEx .....                                              | 54 |
| 5.11.5 | JPEG Capture getJPEG .....                                                                               | 54 |
| 5.11.6 | Set Resolution to Capture BMP Picture getBMPFixPixelEx .....                                             | 55 |
| 5.11.7 | Set Resolution to Capture Picture in Fisheye Mode fecCaptureFixPixel .....                               | 55 |
| 5.11.8 | Get Size of Picture Captured with Specified Resolution in Fisheye Mode<br>fecGetCapPicSizeFixPixel ..... | 56 |
| 5.11.9 | Capture JPEG Picture in Hardware Decoding Mode getHDJPEG .....                                           | 56 |
| 5.12   | Hardware Decoding .....                                                                                  | 58 |
| 5.12.1 | Set Hardware Decoding Priority setHardDecode .....                                                       | 58 |
| 5.12.2 | Set Max. Hardware Decoding Channel Number setMaxHardDecodePort .....                                     | 59 |
| 5.12.3 | Get Current Decoding Mode getDecoderType .....                                                           | 59 |
| 5.12.4 | Switch to Software Decoding switchToSoft .....                                                           | 60 |
| 5.12.5 | Switch to Hardware Decoding switchToHard .....                                                           | 60 |
| 5.12.6 | Set Whether to Reset Hardware Decoding when Exception Occurs<br>setResetHardDecodeFlag .....             | 61 |
| 5.13   | Pre-recording .....                                                                                      | 62 |
| 5.13.1 | Set Pre-recording Data Callback setPreRecordCallBack .....                                               | 62 |
| 5.13.2 | Set Pre-recording Data Callback setPreRecordCallBackEx .....                                             | 63 |
| 5.13.3 | Set Pre-recording Status SetPreRecordFlag .....                                                          | 64 |
| 5.14   | Fisheye .....                                                                                            | 66 |
| 5.14.1 | Set Image Correction setImageCorrection .....                                                            | 66 |
| 5.14.2 | Set Fisheye Dewarping Type setFECDisplayEffect .....                                                     | 66 |

|                                           |                                                                                                    |    |
|-------------------------------------------|----------------------------------------------------------------------------------------------------|----|
| 5.14.3                                    | Get Fisheye Dewarping Parameter getFECDisplayParam.....                                            | 67 |
| 5.14.4                                    | Set Fisheye Dewarping Parameter setFECDisplayParam.....                                            | 67 |
| 5.14.5                                    | Enable Fisheye Dewarping enableFEC .....                                                           | 68 |
| 5.14.6                                    | Disable Fisheye Dewarping disableFEC.....                                                          | 68 |
| 5.14.7                                    | Get Fisheye Dewarping Sub Port getPortFEC.....                                                     | 69 |
| 5.14.8                                    | Delete Fisheye Dewarping Sub Port delPortFEC .....                                                 | 71 |
| 5.14.9                                    | Set Fisheye Dewarping Parameters setParamFEC.....                                                  | 71 |
| 5.14.10                                   | Get Fisheye Dewarping Parameters getParamFEC.....                                                  | 72 |
| 5.14.11                                   | Set Displaying Window setWnd .....                                                                 | 72 |
| 5.14.12                                   | Set Displaying Window setWndEx .....                                                               | 73 |
| 5.14.13                                   | Get Current Selected Fisheye Sub Port getCurrentPTZPort .....                                      | 74 |
| 5.14.14                                   | Set Current Selected Fisheye Sub Port setCurrentPTZPort .....                                      | 74 |
| 5.14.15                                   | Select by Drawing Rectangle or Irregular Frame setPTZoutLineShowMode.....                          | 75 |
| 5.14.16                                   | Get PTZ Coordinates of Current Sub Port setPTZ2Window .....                                        | 75 |
| 5.14.17                                   | Get PTZ Coordinates of Current Sub Port setFECDisplayPTZ.....                                      | 76 |
| 5.14.18                                   | Set Rotation Parameters of 3D Fisheye Dewarping setFish3DRotate .....                              | 77 |
| 5.14.19                                   | Get Cache Size for Fisheye Capture getFishCapPicSize .....                                         | 79 |
| 5.14.20                                   | Capture in Fisheye Mode getFishCapPic.....                                                         | 79 |
| 5.14.21                                   | Set Fisheye Capture Callback Function setFECDisplayCB.....                                         | 79 |
| 5.14.22                                   | Set Absolute Rotation Parameters of 3D Fisheye Dewarping<br>setFish3DRotateAbs.....                | 80 |
| 5.14.23                                   | Get Absolute Rotation Parameters of 3D Fisheye Dewarping<br>getFish3DRotate .....                  | 80 |
| 5.14.24                                   | Get Optimal Fisheye View Parameters (Wall Mounting)<br>getFish3DRotateSpecialView.....             | 81 |
| 5.14.25                                   | Set Animation for 3D Fisheye setAnimation .....                                                    | 81 |
| 5.14.26                                   | Set Fisheye Dewarping Parameters for EZVIZ Camera SetLDCFlag .....                                 | 82 |
| 5.15                                      | Private Data .....                                                                                 | 83 |
| 5.15.1                                    | Display Private Data renderPrivateData .....                                                       | 83 |
| 5.15.2                                    | Display Private Data renderPrivateDataEx .....                                                     | 83 |
| 5.15.3                                    | Set Private Data Callback Function setAdditionalCallback.....                                      | 84 |
| 5.15.4                                    | Set Private Data Callback Function setIVSDrawFunCallback.....                                      | 85 |
| 5.15.5                                    | Set Font Library Path setOverlayPriInfoFlag.....                                                   | 86 |
| 5.15.6                                    | Set the Background Frame Color of the Private Data with POS Information<br>setPosBGRectColor ..... | 87 |
| 5.16                                      | Reverse Playing .....                                                                              | 88 |
| 5.16.1                                    | Play Reversely ReversePlay .....                                                                   | 88 |
| 5.17                                      | Others .....                                                                                       | 88 |
| 5.17.1                                    | Set Squeak Configurations setHSParam .....                                                         | 88 |
| 5.17.2                                    | Set Callback Function of PCM Audio Data setAudioDataCallBack .....                                 | 89 |
| Chapter 6 Structure and Enumeration ..... |                                                                                                    | 89 |
| 6.1                                       | SRTRANS_PARAM .....                                                                                | 89 |
| 6.2                                       | SRTRANS_ELEMENT.....                                                                               | 90 |
| 6.3                                       | FISHEYE_PARAM .....                                                                                | 90 |

|                    |                                                                                                                               |    |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------|----|
| 6.4                | PTZ_PARAM .....                                                                                                               | 91 |
| 6.5                | CYCLE_PARAM .....                                                                                                             | 91 |
| 6.6                | PRIDATA_RENDER .....                                                                                                          | 91 |
| 6.7                | FIRE_ALARM .....                                                                                                              | 92 |
| 6.8                | TEM_FLAG .....                                                                                                                | 92 |
| Chapter 7 FAQ..... |                                                                                                                               | 94 |
| 7.1                | Stream Mode .....                                                                                                             | 94 |
| 7.1.1              | Why does the error occur when transferring parameters to <i>PlayM4_OpenStream</i> ? .....                                     | 94 |
| 7.1.2              | How to perform fast forward when playing in stream mode? .....                                                                | 94 |
| 7.1.3              | Why does the first frame display slower than other frames in the live view? .....                                             | 94 |
| 7.2                | Live View .....                                                                                                               | 95 |
| 7.2.1              | How to solve the struck phenomenon during the live view? .....                                                                | 95 |
| 7.2.2              | Why is there no image displayed after starting live view? .....                                                               | 95 |
| 7.2.3              | How to solve the delay problem of live view? .....                                                                            | 96 |
| 7.3                | Encrypted Stream .....                                                                                                        | 96 |
| 7.3.1              | Why can't I play the encrypted stream? .....                                                                                  | 96 |
| 7.4                | Capture .....                                                                                                                 | 96 |
| 7.4.1              | How to save the captured picture in the memory for following processing? .....                                                | 96 |
| 7.5                | Play Audio .....                                                                                                              | 97 |
| 7.5.1              | Why is it stuck when playing audio, but fluent when playing video? .....                                                      | 97 |
| 7.5.2              | Why is there no sound when playing file? .....                                                                                | 97 |
| 7.6                | File Mode.....                                                                                                                | 97 |
| 7.6.1              | Why does the fast forward occur when playing file? .....                                                                      | 97 |
| 7.6.2              | Failed to open files.....                                                                                                     | 97 |
| 7.6.3              | How to locate in file mode? .....                                                                                             | 98 |
| 7.7                | Decoding Callback.....                                                                                                        | 98 |
| 7.7.1              | How to get the decoded data of multiple channels? .....                                                                       | 98 |
| 7.7.2              | How to avoid frame loss when calling decoding callback function in live view? .....                                           | 98 |
| 7.7.3              | Why does the video frame obtained from decoding callback function only display a half height? .....                           | 98 |
| 7.7.4              | What's the difference between the decoding callback function and display callback function? .....                             | 99 |
| 7.7.5              | Why is the timestamp information repeated when continuously getting three audio data in the decoding callback function? ..... | 99 |
| 7.8                | Synchronous Playback .....                                                                                                    | 99 |
| 7.8.1              | How to realize the synchronous playback of multiple files? .....                                                              | 99 |
| 7.9                | Others .....                                                                                                                  | 99 |
| 7.9.1              | Failed to get the live view .....                                                                                             | 99 |

# Chapter 1 Overview

## 1.1 Introduction

The Mobile Platform (Android) PlayCtrl Library SDK is the secondary development kit for the decoding of HIKVISION DVR, DVS and IP devices, etc. It implements the functions of live view, playback, playing control (such as pause, single frame forward/backward, etc), getting stream information (e.g., file indexing information, decoding information, resolution, frame rate, etc), capturing pictures in BMP or JPG format, and so on.

## 1.2 Product Scopes

- NVR: DS-95xx/96xx series and DS-76xx series.
- Hybrid DVR: DS-90xx series and DS-76xx series.
- DVR, ATM DVR, Mobile DVR: DS-91xx series, DS-81xx/71xx/72xx series, DS-80xx/70xx series, DS-73xx series.
- DVS, Decoder, Encoder: DS-60xx series, DS-61xx series, DS-63xx series, DS-64xx series, DS-65xx series, and DS-66xx.
- Compression Card: DS-40xx/41xx/42xx/43xx series.
- Network Camera, Network Speed Dome, etc.

## 1.3 Conventions

To simplify the descriptions, the Mobile Platform (Android) PlayCtrl Library SDK hereby is referred to as “SDK” or “PlayCtrl Library SDK” in the following contents.

## Chapter 2 Update History

### Version Description:

The naming rules of Mobile Platform (Android) PlayCtrl Library SDK version is described as follows.

*V Main Version. Sub Version. Fix Version. Reserved Version*

- **Main Version:** Large-scale modification, re-construction or optimization of the SDK
- **Sub Version:** Additional functions/features added to the SDK
- **Fix Version:** Partial changes or bug-fixing of the SDK
- **Reserved Version:** Reserved

**Note:** If your CPU supports Hyper-Threading Technology, it is recommended to use V3.4 or higher version of the SDK.

### 2.1 Version 7.4.0.X (2020.09)

- Provided new version of IDEMUX library.
- Provided new version of decoding library.
- Added function of anti-aliasing.
- Added function of image post-processing.
- Added function of audio noise reduction.
- Combined hardware decoding and software decoding of fisheye module.
- Optimized the delay of hardware decoding and added speed-up solution of hardware decoding.
- Added function of setting image post-processing parameters, related API: [setImagePostProcessParameter](#).
- Added function of setting background frame color of the private data with POS information, related API: [setPosBGRectColor](#).
- Added function of setting whether to reset hardware decoding when exception occurs, related API: [setResetHardDecodeFlag](#).
- Added function of setting ANR algorithm parameters, related API: [setANRParam](#).

### 2.2 Version 7.3.9.X (2020.01)

- Added function of enabling feature tracking effect (for EZVIZ Client), related API: [enableSuperEyeEffect](#).
- Added function of disabling feature tracking effect (for EZVIZ Client), related API: [disableSuperEyeEffect](#).
- Added function of getting parameters of current display area, related API: [getCurrentRegionRect](#).
- Supports multi-screen play in hardware decoding mode.
- Supports multi-screen fisheye dewarping in hardware decoding mode.
- Supports digital zoom effect of fisheye screen in both hardware and software decoding modes (currently only Dual-180° panorama, 360° panorama, and 180° panorama fisheye dewarping)

modes are supported).

- Supports private data display in hardware decoding mode.
- Optimized JNI conversion module.
- Optimized codes of fisheye dewarping and display modules.

## 2.3 Version 7.3.8.X (2019.07)

- Added function of setting expected frame rate to drop frame in case of bad decoding performance, related API: [setExpectedFrameRate](#).
- Added function of setting number of decoding threads, related API: [setDecodeThreadNumber](#).
- Added function of setting resolution to capture BMP picture, related API: [getBMPFixPixelEx](#).
- Added function of setting resolution to capture picture in fisheye mode, related API: [fecCaptureFixPixel](#).
- Added function of getting size of picture captured with specified resolution in fisheye mode, related API: [fecGetCapPicSizeFixPixel](#).
- Added function of getting JPEG picture in hardware decoding mode, related API: [getHDJPEG](#).
- Supports playing AAC\_LD and OPUS audio.
- Supports transmitting stream and playing video after enabling hardware decoding mode.
- Supports transmitting stream header from external and resetting corresponding module inside the playing library separately.

## 2.4 Version 7.3.7.X (2018.12)

- Added two audio playing modes: STREAM\_MUSIC and STREAM\_VOICE\_CALL, related API: [SetAudioTrackParam](#).
- Added function of setting absolute timestamp for video playing, related API: [setAbsTimeFlag](#).
- Added function of setting AGC parameters for audio, related API: [setAGCParam](#).
- Added function of setting fisheye dewarping parameters for EZVIZ camera, related API: [setLDCFlag](#).
- Added function of setting chunk size for playing RTMP stream, related API: [setDemuxParam](#).
- Supports playing stream encrypted by HIK algorithm.
- Supports overlaying private data on picture.
- Added fisheye original view mode.
- Added context sharing mode via Android EGL.
- Edited EGL binding mechanism.
- Canceled distinguishing ceil mounting and ground mounting between semisphere view and asteroids view of fisheye dewarping.
- Fixed the bug of security risk in the previous version.
- Fixed the fisheye dewarping bug of EZVIZ cameras.
- Fixed the initial position bug when enabling fisheye dewarping.

## 2.5 Version 7.3.5.X (2018.04)

- Added function of setting decoding tolerance level, related API: [setDecodeERC](#).
- Added a fisheye dewarping mode: FEC\_CORRECT\_ARC\_VERTICAL-180° panorama+vertical curved surface (wall mounting), related API: [getPortFEC](#).
- Added function of setting fisheye display callback function, related API: [setFECDisplayCB](#).
- Added function of playing Intra flash stream.
- Added the external management mechanism for reference frame.

## 2.6 Version 7.3.5.X (2017.11)

- Added fishsye function in hardware decoding mode.
- Added software decoding mechanism without copying YUV.
- Added 3D fisheye dewarping modes: FEC\_CORRECT\_ARC\_HORIZONTAL, related API: [getPortFEC](#)

## 2.7 Version 7.3.4.X (2017.03)

### Added:

- Added display callback (with global time): [setDisplayCBEx](#).
- Added decoding callback (with global time): [setDecodeCBEx](#).
- Set pre-record callback (with global time): [setPreRecordCallBackEx](#);
- Added APIs for enabling or disabling audio sharing: [playSoundShare](#), [stopSoundShare](#).
- Added API for setting fisheye 3D rotation: [setFish3DRotate](#).
- Added APIs for fisheye capture: [getFishCapPicSize](#), [getFishCapPic](#).
- Added API for BMP capture of private information: [getBmpEx](#).
- Added API for reverse play: [reversePlay](#).

### Edited:

- Four types of 3D effects are added in [getPortFEC](#) to the Fisheye Module, including FEC\_CORRECT\_SEM (Hemisphere), FEC\_CORRECT\_CYC (Cylinder), FEC\_CORRECT\_PLA (Asteroid) and FEC\_CORRECT\_CYC\_SPL (Cylinder Extended to Plane).

## 2.8 Version 7.3.3.X (2016.10)

### Added:

- Regulating the Sound Volume Dynamically Version 7.3.2.X (2016.06)

### Added:

- Add API of multi-screen PTZ correction of fisheye: [enableFEC](#), [disableFEC](#), [getPortFEC](#), [delPortFEC](#), [setParamFEC](#), [getParamFEC](#), [setWnd](#), [setWndEx](#), [getCurrentPTZPort](#), [setCurrentPTZPort](#),

[setPTZoutLineShowMode](#), [setPTZ2Window](#).

- Add API of privacy data (POS information overlay and picture overlay are not supported): [renderPrivateDataEx](#), [renderPrivateData](#).
- Add API of synchronous playing audio and video: [syncToAudio](#).
- Add API of setting window transparency: [setWindowTransparency](#).

## Chapter 3 Error Code

### 3.1 Common Error Code

| ID                           | Code | Description                                                        |
|------------------------------|------|--------------------------------------------------------------------|
| PLAYM4_NOERROR               | 0    | No error                                                           |
| PLAYM4_PARA_OVER             | 1    | Illegal input parameter                                            |
| PLAYM4_ORDER_ERROR           | 2    | Calling reference error                                            |
| PLAYM4_TIMER_ERROR           | 3    | Set timer failure                                                  |
| PLAYM4_DEC_VIDEO_ERROR       | 4    | Video decoding failure                                             |
| PLAYM4_DEC_AUDIO_ERROR       | 5    | Audio decoding failure                                             |
| PLAYM4_ALLOC_MEMORY_ERROR    | 6    | Memory allocation failure                                          |
| PLAYM4_OPEN_FILE_ERROR       | 7    | File operation failure                                             |
| PLAYM4_CREATE_OBJ_ERROR      | 8    | Create thread failure                                              |
| PLAYM4_BUF_OVER              | 11   | Buffer overflow, input stream failure                              |
| PLAYM4_CREATE_SOUND_ERROR    | 12   | Create sound device failure                                        |
| PLAYM4_SET_VOLUME_ERROR      | 13   | Set volume failure                                                 |
| PLAYM4_SUPPORT_FILE_ONLY     | 14   | This API can only be called in file decoding mode                  |
| PLAYM4_SUPPORT_STREAM_ONLY   | 15   | This API can only be called in stream decoding mode                |
| PLAYM4_SYS_NOT_SUPPORT       | 16   | System not support, the SDK can only work with CPU above Pentium 3 |
| PLAYM4_FILEHEADER_UNKNOWN    | 17   | Missing file header                                                |
| PLAYM4_VERSION_INCORRECT     | 18   | Version mismatch between encoder and decoder                       |
| PLAYM4_INIT_DECODER_ERROR    | 19   | Initialize decoder failure                                         |
| PLAYM4_CHECK_FILE_ERROR      | 20   | File too short or unrecognizable stream                            |
| PLAYM4_INIT_TIMER_ERROR      | 21   | Initialize timer failure                                           |
| PLAYM4_BLT_ERROR             | 22   | BLT failure                                                        |
| PLAYM4_OPEN_FILE_ERROR_MULTI | 24   | Open video & audio stream failure                                  |
| PLAYM4_OPEN_FILE_ERROR_VIDEO | 25   | Open video stream failure                                          |
| PLAYM4_JPEG_COMPRESS_ERROR   | 26   | JPEG compression failure                                           |
| PLAYM4_EXTRACT_NOT_SUPPORT   | 27   | File type not supported                                            |
| PLAYM4_EXTRACT_DATA_ERROR    | 28   | Data error                                                         |
| PLAYM4_SECRET_KEY_ERROR      | 29   | Secret key error                                                   |
| PLAYM4_DECODE_KEYFRAME_ERROR | 30   | Key frame decoding failure                                         |
| PLAYM4_NEED_MORE_DATA        | 31   | Not enough data                                                    |
| PLAYM4_INVALID_PORT          | 32   | Invalid port number                                                |
| PLAYM4_NOT_FIND              | 33   | No resource found.                                                 |

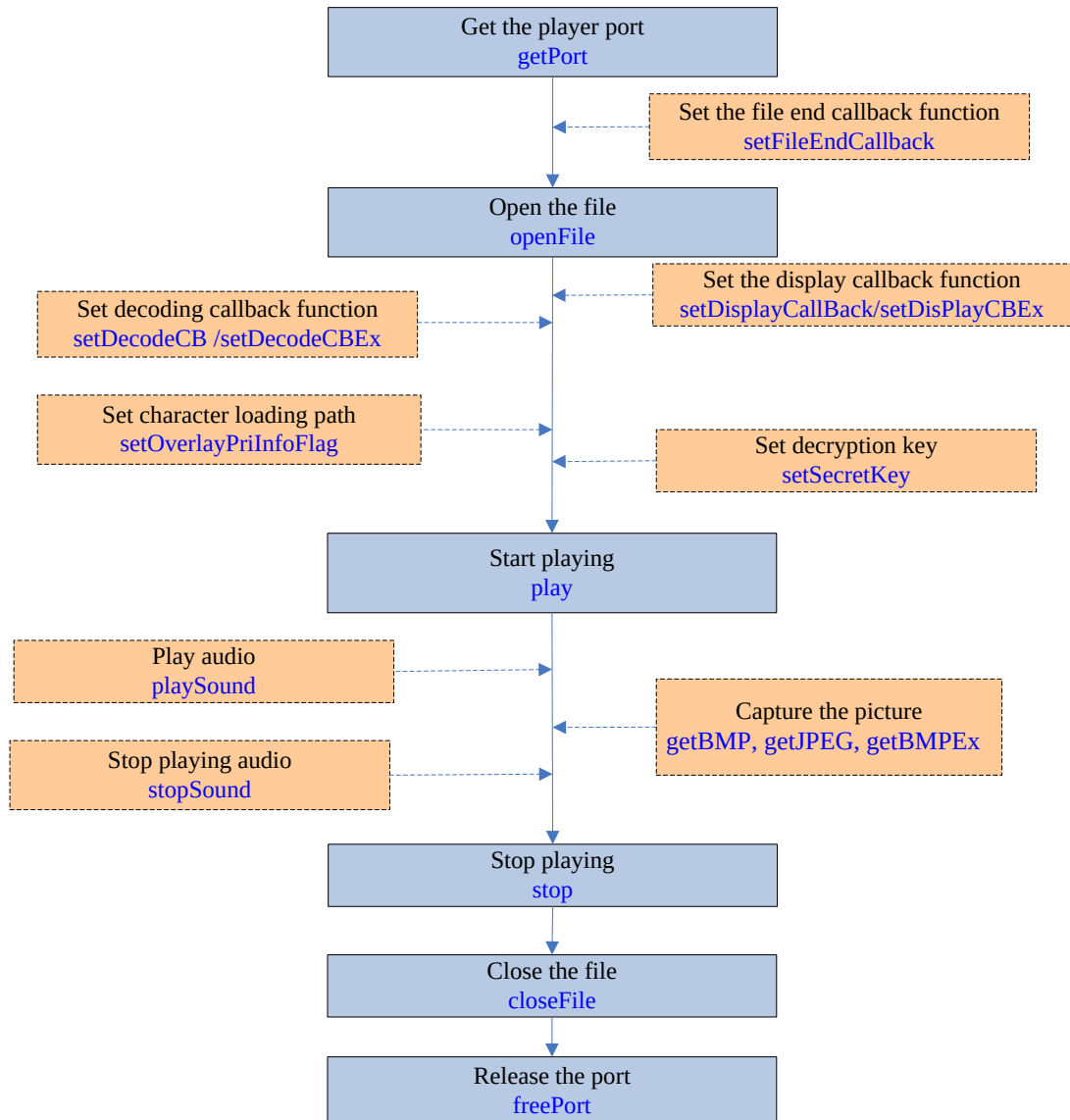
|                           |    |                           |
|---------------------------|----|---------------------------|
| PLAYM4_NEED_LARGER_BUFFER | 34 | Insufficient buffer size. |
| PLAYM4_DEMUX_ERROR        | 35 | Demux error.              |
| PLAYM4_FAIL_UNKNOWN       | 99 | Unknown error.            |

## 3.2 Fisheye Error Code

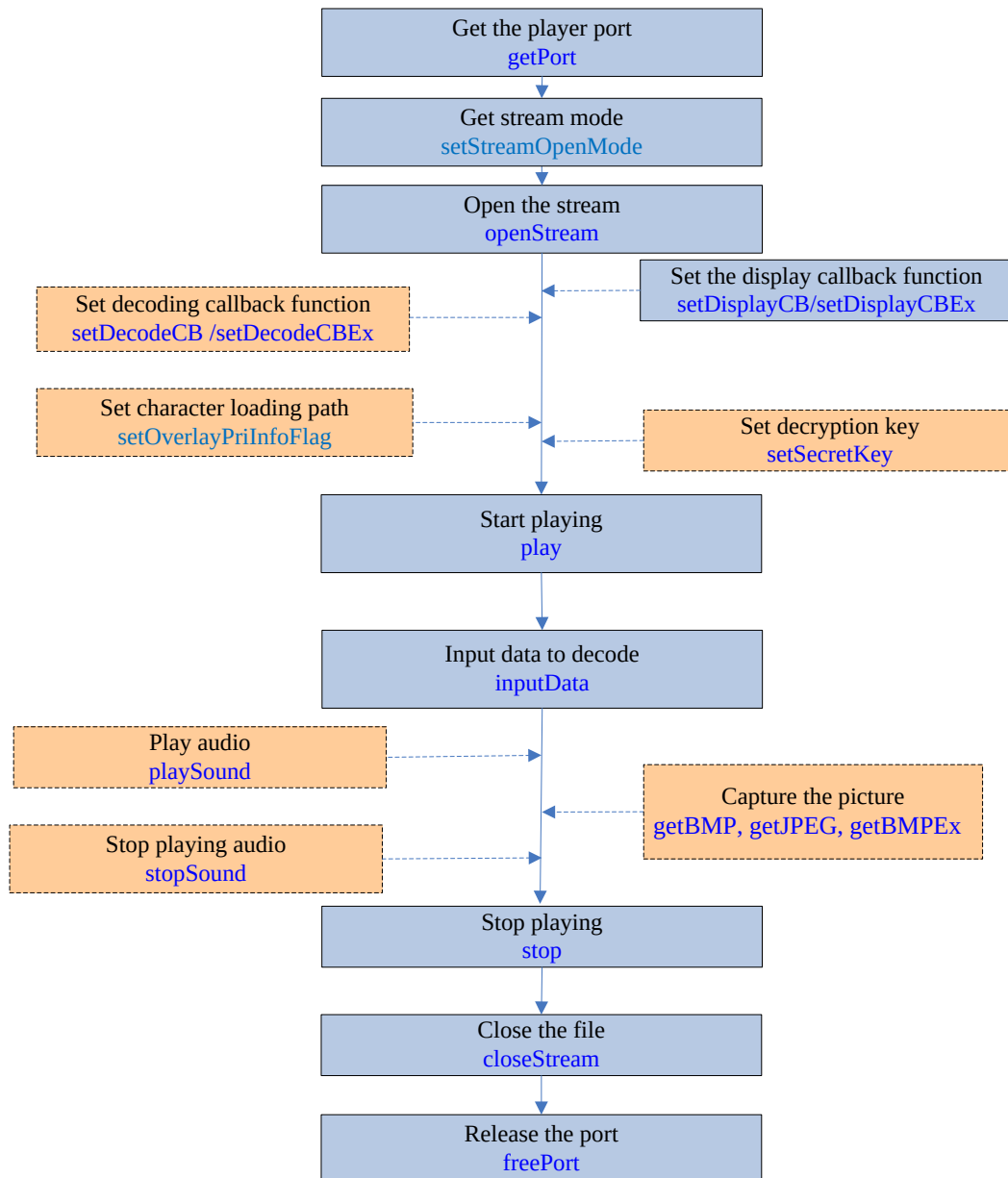
| Type of Error                   | Value | Error Message                                                                                |
|---------------------------------|-------|----------------------------------------------------------------------------------------------|
| PLAYM4_FEC_ERR_ENABLEFAIL       | 100   | Failed to load fisheye module.                                                               |
| PLAYM4_FEC_ERR_NOTENABLE        | 101   | Fisheye module is not loaded.                                                                |
| PLAYM4_FEC_ERR_NOSUBPORT        | 102   | The sub port is not allocated.                                                               |
| PLAYM4_FEC_ERR_PARAMNOTINIT     | 103   | The port parameters are not initialized.                                                     |
| PLAYM4_FEC_ERR_SUBPORTOVER      | 104   | Sub ports have been used up.                                                                 |
| PLAYM4_FEC_ERR_EFFECTNOTSUPPORT | 105   | Not support this effect with this mounting type.                                             |
| PLAYM4_FEC_ERR_INVALIDWND       | 106   | Invalid Window.                                                                              |
| PLAYM4_FEC_ERR_PTZOVERFLOW      | 107   | PTZ positions overflow                                                                       |
| PLAYM4_FEC_ERR_RADIUSINVALID    | 108   | Invalid circle parameters.                                                                   |
| PLAYM4_FEC_ERR_UPDATENOTSUPPORT | 109   | The specified mounting type and correction effect, updating this parameter is not supported. |
| PLAYM4_FEC_ERR_NOPLAYPORT       | 110   | The playing port is disabled.                                                                |
| PLAYM4_FEC_ERR_PARAMVALID       | 111   | The parameter is null.                                                                       |
| PLAYM4_FEC_ERR_INVALIDPORT      | 112   | Invalid sub window                                                                           |
| PLAYM4_FEC_ERR_PTZZOOMOVER      | 113   | PTZ correction area overflow                                                                 |
| PLAYM4_FEC_ERR_OVERMAXPORT      | 114   | Correction channels are used up, up to 4 correction channels are supported.                  |
| PLAYM4_FEC_ERR_ENABLED          | 115   | This port has enabled fisheye module.                                                        |
| PLAYM4_FEC_ERR_D3DACCENOTENABLE | 116   | D3D acceleration is disabled.                                                                |
| PLAYM4_FEC_ERR_PLACE_TYPE       | 117   | Incorrect mounting type. Each playing port should corresponds to one kind of mounting type.  |
| PLAYM4_FEC_ERR_CORRECT_TYPE     | 118   | Incorrect dewarping type.                                                                    |

## Chapter 4 API Call Flow

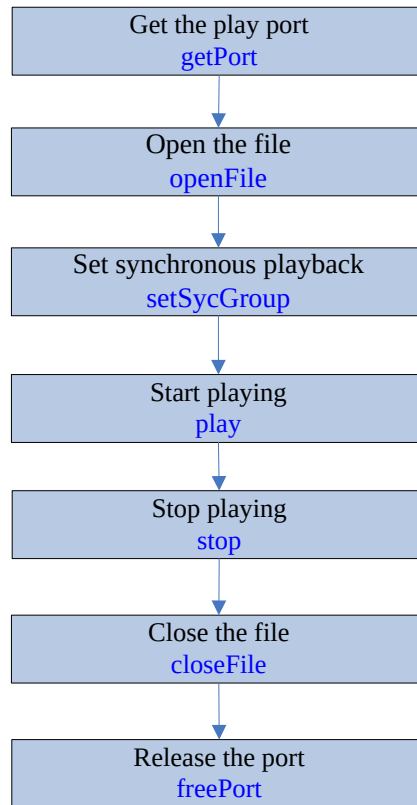
### 4.1 Play in File Mode



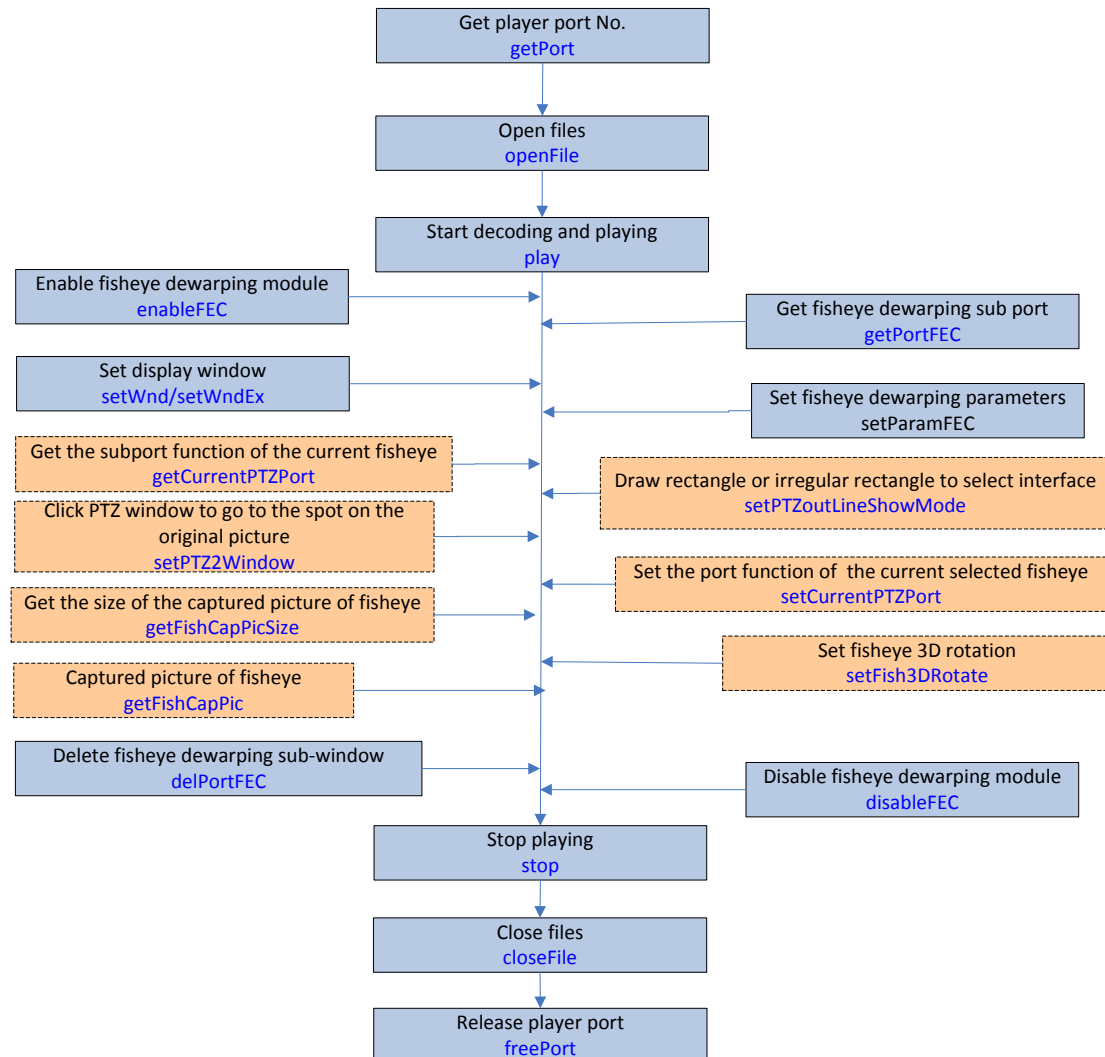
## 4.2 Play in Stream Mode



## 4.3 Synchronous Playback



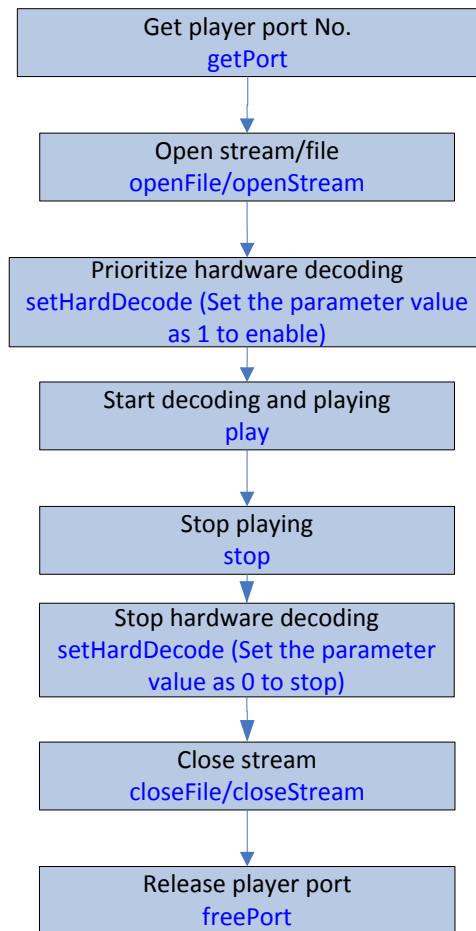
## 4.4 Fisheye



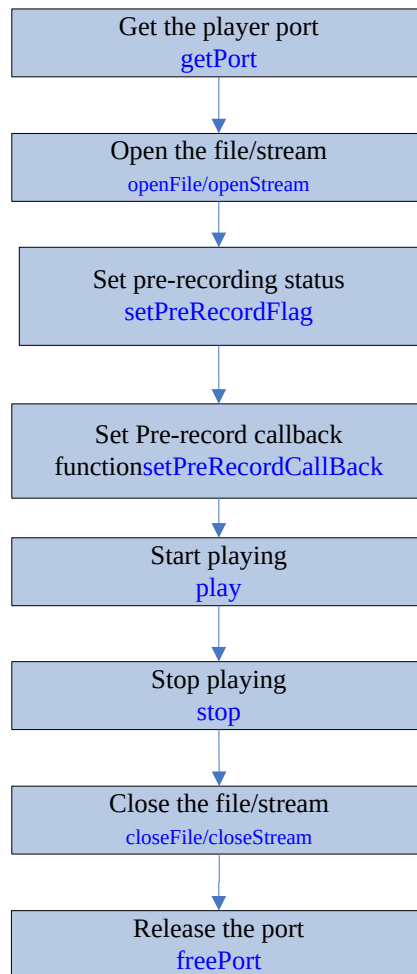
The fisheye module provides the image dewarping of fisheye camera. Before using this module, you should enable live view module, get fisheye dewarping sub port, set fisheye dewarping parameters, set display window and set drawing callback.

- Call API `enableFEC` or `disableFEC` API to enable or disable fisheye module
- Call API `getPortFEC` or `delPortFEC` to get or delete fisheye dewarping sub port.
- Call API `setParamFEC` or `getParamFEC` to set or get fisheye dewarping parameters
- The fisheye APIs of version 7.3.1 or previous and version 7.3.2 or later cannot be called at the same time. Fisheye API of version 7.3.2 or later is suggested.
- Call API `setWnd` or `setWndEx` to set display window.
- Before calling `getFishCapPic` to realize fisheye capture, you should call `getFishCapPicSize` to get picture size.
- The above flow chart is based on file mode. While the stream mode also supports fisheye module.
- Fisheye display window cannot be recreated if it is destroyed.

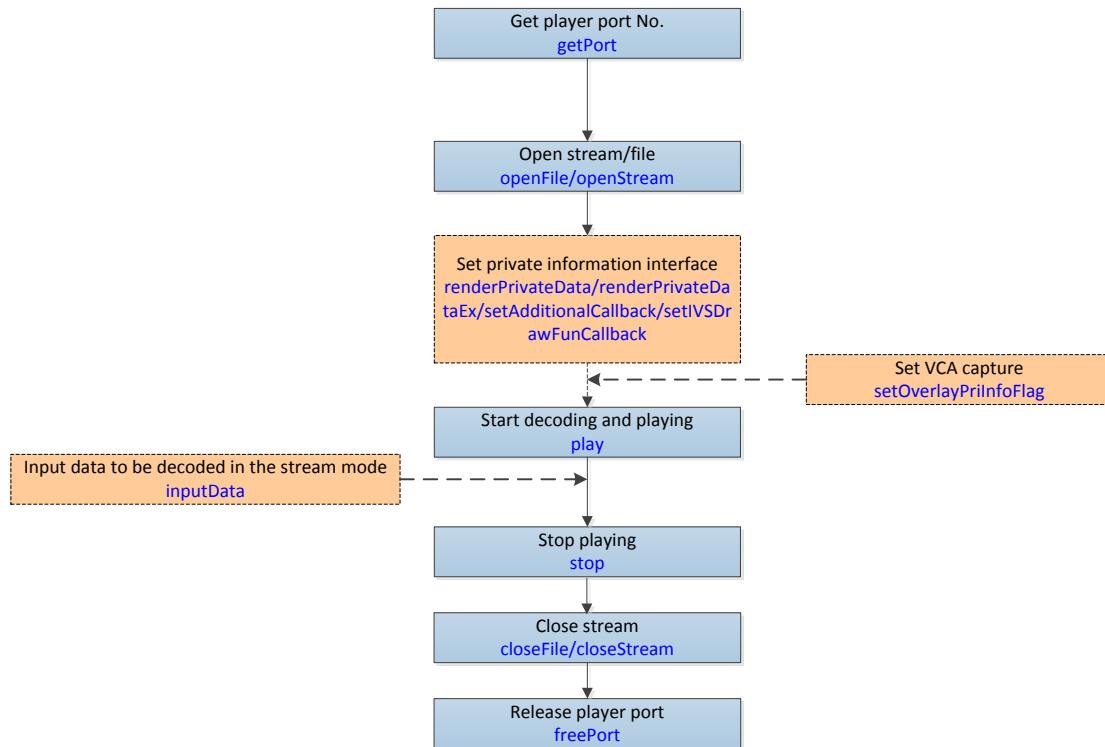
## 4.5 Hardware Decoding



## 4.6 Pre-recording



## 4.7 Display Private Information



- Private Information Module: It is applicable to the stream with VCA information. Otherwise, there is no effect to call this API.
- Set the display of private information (major type: VCA, motion detection, POS information overlay, picture overlay, thermal imaging information): renderPrivateData.
- Set the display of private information (minor type: when nIntelType is PLAYM4\_RENDER\_FIRE\_DETCT, the value is PLAYM4\_FIRE\_ALARM; when nIntelType is PLAYM4\_RENDER\_TEM, the value is PLAYM4\_TEM\_FLAG).
- Set private information drawing callback: setIVSDrawFunCallback.
- Call renderPrivateData, renderPrivateDataEx, setAdditionalCallback, setIVSDrawFunCallback before or after calling PlayM4\_Play; but call PlayM4\_SetOverlayPriInfoFlag after the image is decoded, and after calling PlayM4\_Play.
- Set capture with private information: setOverlayPriInfoFlag.

## Chapter 5 API

### 5.1 System Operation and Error Code

#### 5.1.1 Get SDK Version No. and Build No. **getSdkVersion**

```
int getSdkVersion();
```

**Description:**

Get SDK version and build number.

**Parameters:**

--

**Returned Value:**

The higher 16 digits stands for the current build number, digits 9~16 stands for the main version and digit 1~8 is the sub version, e.g. return value 0x06040105 stands for Version1.5, Build 0604.

**Notice:**

For the debug version SDKs, there will only be difference in build number.

#### 5.1.2 Get Error Code **getLastError**

```
int getLastError(int nPort);
```

**Description:**

Get the error code.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

The value specified the error code. For the error description, please refer to the table in Chapter 3.

#### 5.1.3 Get Port Number **getPort**

```
int getPort();
```

**Description:**

Get a valid port number, and the port number should be in the range of [0, 32).

**Parameters:**

**nPort**

[in] [out] pointer of get the port number

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

### 5.1.4 Release Playing Port **freePort**

```
boolean freePort(int nPort);
```

**Description:**

Release the port number which has been occupied.

**Parameters:**

**nPort**

[in] Playing port number

It is suggested to set the **nPort** as -1 after a successful port releasing.

**Returned Value:**

true - API calling succeeds;

false - API calling fails

## 5.2 File Operation

### 5.2.1 Open File **openFile**

```
boolean openFile(int nPort, String sFilePath);
```

**Description:**

Open the file for playback.

**Parameters:**

**nPort**

[in] Playing port number

**sFileName**

[in] The file name

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

- The file size ranges from 4KB to 4GB.
- Only support files stored locally.

### 5.2.2 Close File **closeFile**

```
boolean closeFile(int nPort);
```

**Description:**

Close the file that has been opened.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

### 5.2.1 Set File Index Callback **setFileRefCB**

```
boolean setFileRefCB(int nPort, PlayerFileRefCB fileRefCB);
```

**Description:**

Register a callback function to notify the user when the key frame index for the file is created. If the callback is not triggered, it indicates errors in the file.

The file index is used for fast locating in the file. All the Index-related APIs should be called after indexing, and the other APIs will not be affected.

**Parameters:**

**nPort**

[in] Playing port number

**fileRefCB**

[in] pointer of callback function

**Description of the Callback Function:**

```
public void onFileRefDone(int nPort);
```

**Parameters:**

**nPort**

*Playing port number*

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

## 5.3 Stream Operation

### 5.3.1 Set Stream Input Mode **setStreamOpenMode**

```
boolean setStreamOpenMode(int nPort, int nMode);
```

**Description:**

Set stream input mode.

**Parameters:**

**nPort**

[in] Playing port number

**nMode**

[in] work in stream mode:

**0: STREAME\_REALTIME** mode (default)

**1: STREAME\_FILE** mode (play by time stamp)

**STREAME\_REALTIME** mode gives priority to ensuring of real-time performance and preventing of data blocking problem, and is with strict data checking mechanism while **STREAME\_FILE** is the contrary.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remark:**

This API should be called before playback starts.

### 5.3.2 Open Stream File **openStream**

```
boolean openStream(int nPort, byte[] pFileHeadBuf, int nSize, int nBufPoolSize);
```

**Description:**

Open the stream for playback (similar with open file).

**Parameters:**

**nPort**

[in] Playing port number

**pFileHeadBuf**

[in] The file header which is got from the recording callback APIs of network client SDK or card SDK

**nSize**

[in] The data length of the file header.

**nBufPoolSize**

[in] Specify the size of the source buffer. It ranges from SOURCE\_BUF\_MIN to SOURCE\_BUF\_MAX. Users may encounter decoding failure if nBufPoolSize is too small. It is suggested that nBufPoolSize be no less than 200\*1024 for SD (standard definition) devices, and no less than 600\*1024 for HD (high definition) devices.

```
#define SOURCE_BUF_MAX 1024*100000
```

```
#define SOURCE_BUF_MIN 1024*50
```

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

### 5.3.3 Close Stream File **closeStream**

```
int PlayM4_CloseStream (int nPort);
```

**Description:**

Close the stream which has been opened.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

### 5.3.4 Input Stream Data **inputData**

```
boolean inputData(int nPort, byte[] pBuf, int nSize);
```

**Description:**

Input stream data.

**Parameters:**

**nPort**

[in] Playing port number

**pBuf**

[in] buffer address

**nSize**

[in] buffer size

**Returned Value:**

If the data input succeeds, the return value is true.

If the data input fails, the return value is false. Stream data input should starts after OpenStream, and a false return is often caused by the case of full buffer.

**Remarks:**

Suggestions for data input failure handling:

- For the data input failure caused by full buffer under STREAME\_REALTIME mode, users can either discard the data (which will cause frame loss or incomplete decoding video) or retry after sleep of several milliseconds.
- For the data input failure caused by full buffer under STREAME\_FILE mode, users should retry until input succeeds.

### 5.3.5 Set Encrypted Stream Callback Function

#### SetEncryptTypeCallBack

**API Definition:**    `boolean SetEncryptTypeCallBack (`  
                          `int                      nPort,`  
                          `int                      nType,`  
                          `PlayerEncryptTypeCB    EncryptTypeCB`  
                          `)`

**Parameters:**    *nPort*                      Play port No.  
                          *nType*                      Callback type, 0-call back once changed; 1-call back when encryption is detected.

*EncryptTypeCB*    Encrypted stream callback, see details below:

```
public interface PlayerEncryptTypeCB{
    void onEncrypt(
        int    nPort,
        int    nVideoEnctrpyType,
        int    nAudioEnryptType,
        int    nSetSecretKey
    );
}
```

**Callback Function Parameters:**

*nPort*: Port No.

*nVideoEnctrpyType*: Video encryption type

*nAudioEnryptType*: Audio encryption type

*nSetSecretKey*: Encryption tag

**Return Values:**    Return *true* for success, and return *false* for failure.

## 5.4 Playing and Control

### 5.4.1 Start Playing **play**

`boolean play(int nPort, SurfaceHolder holder);`

**Description:**

Start playback. The display image size will be automatically adjusted according to the SurfaceHolder window size. For full screen display, please magnify the SurfaceHolder window to full screen size.

If the video is already in playback status, calling this API will reset the playback speed to 1X.

**Parameters:**

**nPort**

[in] Playing port number

**holder**

[in] The handle of the display window.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

### 5.4.2 Start Playing **playEx**

**Function:** `boolean playEx(int nPort, SurfaceTexture texture)`

**Parameters:**      `int`                      `nPort`                      Playing channel No.  
                         `SurfaceTexture texture`                      Display window

**Returned Value:**      Return true on success, and return false on failure.

**Remarks:**

- The window size can be adjusted according to texture window during playing. If you want to display in full screen, you can extend the texture window to full screen.
- If the recording file has been played, you should reset the current playing speed to normal speed.

**Remark:**                      Parsing may failed if I frame size is larger than 2 MB.

### 5.4.3 Stop Playing **stop**

`boolean stop(int nPort);`

**Description:**

Stop playback.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

true - API calling succeeds;  
false - API calling fails.

### 5.4.4 Pause Playing **pause**

```
boolean pause(int nPort, int nPause);
```

**Description:**

Pause or resume playback.

**Parameters:**

**nPort**

[in] Playing port number

**nPause**

[in] 1: pause, 0: resume the previous playback process

**Returned Value:**

true - API calling succeeds;  
false - API calling fails.

### 5.4.5 Fast Forward **fast**

```
boolean fast(int nPort);
```

**Description:**

Fast forward. This API can be called up to 4 times continuously to increase the playback speed, which will be doubled after each API call. Call play() to restore 1X playback from the current position.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

true - API calling succeeds;  
false - API calling fails.

**Remark:**

HD video may not reach the fast forward speed set by the user due to limitation of decoding and display performance.

### 5.4.6 Slow Forward **slow**

```
boolean slow(int nPort);
```

**Description:**

Slow forward. This API can be called up to 4 times continuously to decrease the playback speed, which will be lowered by half after each API call. Call play() to restore 1X playback from the

current position.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

## 5.4.7 Play Audio in Exclusive Mode **playSound**

```
boolean playSound(int nPort);
```

**Description:**

Open the audio. Only 1-ch audio playback can be enabled, and the audio on other channels will be switched off automatically.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

- The audio display is disabled by default.
- playSound() and stopSound() should be used together in the application. If playSound() is called, then stopSound() should be called before application ends.
- When playing low frame rate file, please enable the audio before playing, otherwise it may cause audio playing exception.

## 5.4.8 Stop Playing Audio in Exclusive Mode **stopSound**

```
boolean stopSound();
```

**Description:**

Stop the audio playback.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

## 5.4.9 Play Audio in Sharing Mode **playSoundShare**

```
Function:      boolean playSoundShare(  
                int    nPort,  
                );
```

Parameter: nPort

[in] Playing channel No.

**Returned Value:** Return *true* on success, and return *false* on failure.

**Remarks:**

- This API is used to play the audio in sharing mode.
- `playSoundShare` and `stopSoundShare` should be called by pair.
- For G series sound sampling rate, play the audio according to the data on encapsulation layer. If the sampling rate of files or stream set on the encapsulation layer is not correct, playing exception may occur.
- Before playing file with low frame rate, the sound should be enabled. Otherwise, playing exception may occur.
- If there is no audio encoding type defined in the 40-byte file header of `openStream`, playing error may occur.
- If this API is successfully called when there is no audio data in the stream, no sound will be played.

**Remark:** By default, the audio data will not be handled if this API is not called.

## 5.4.10 Stop Playing Audio in Sharing Mode `stopSoundShare`

**Function:**

```
boolean stopSoundShare(  
    int    nPort,  
);
```

**Parameter:** `nPort`  
[in] Playing channel No.

**Returned Value:** Return *true* on success, and return *false* on failure.

**Remarks:**

- `playSoundShare` and `stopSoundShare` should be called by pair.
- For G series sound sampling rate, play the audio according to the data on encapsulation layer. If the sampling rate of files or stream set on the encapsulation layer is not correct, playing exception may occur.
- Before playing file with low frame rate, the sound should be enabled. Otherwise, playing exception may occur.
- If there is no audio encoding type defined in the 40-byte file header of `openStream`, playing error may occur.
- If this API is successfully called when there is no audio data in the stream, no sound will be played.

## 5.4.11 Adjust Volume `adjustWaveAudio`

```
boolean adjustWaveAudio(int nPort, int nValue);
```

**Parameters:**

**nPort**

[in] Playing port number

**nValue**

[in] adjusting parameters, "0" stand for "do not adjust", ranging from -100 to 100.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Description:**

Regulate the sound volume dynamically when the audio is turned on, the range of nValue is -100 to 100.

**Remark:**

If the sound volume of the system is too small, the volume change would be not obvious by using the adjustWaveAudio.

## 5.4.12 Set Playing Progress (Percentage) **setPlayPos**

```
boolean setPlayPos(int nPort, float fRelativePos);
```

**Description:**

Locate the relative playback position of the file (percentage). To get precise locating performance, please create file index before executing this operation, otherwise it can only achieve rough locating.

**Parameters:****nPort**

[in] Playing port number

**fRelativePos**

[in] The percent of the file relative position, ranging from 0% to 100%.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

## 5.4.13 Get Playing Progress (Percentage) **getPlayPos**

```
float getPlayPos(int nPort);
```

**Description:**

Get the relative playback position of file (percentage).

**Parameters:****nPort**

[in] Playing port number

**Returned Value:**

The percentage of the file's relative playback position, ranging from 0% to 100%.

## 5.4.14 Set Played Time (ms) **setPlayedTimeEx**

```
boolean setPlayedTimeEx(int nPort, int nTime);
```

**Description:**

Set the new position in the file in milliseconds for playback. To get precise locating performance, users need to create file index before executing this operation, otherwise it can only achieve

rough locating.

**Parameters:**

**nPort**

[in] Playing port number

**nTime**

[in] Start time for playback

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

### 5.4.15 Get Played Time (ms) **getPlayedTimeEx**

```
long getPlayedTimeEx (int nPort);
```

**Description:**

Get the current playback position of the file in milliseconds.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

The current playback position of the file (unit: millisecond)

### 5.4.16 Set Playing Progress (Frame) **setCurrentFrameNum**

```
boolean setCurrentFrameNum(int nPort, int nFrameNum);
```

**Description:**

Specifies the playback position in the file (unit: frames) from the beginning. To get precise locating performance, users need to create file index before executing this operation, otherwise it can only achieve rough locating.

**Parameters:**

**nPort**

[in] Playing port number

**nFrameNum**

[in] The starting frame number for playback

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

### 5.4.17 Get Playing Progress (Frame) **getCurrentFrameNum**

```
Int getCurrentFrameNum(int nPort);
```

**Description:**

Get the current playback position in the file in frame number from the beginning

**Parameters:****nPort**

[in] Playing port number

**Returned Value:**

The current frame number for playback

## 5.4.18 Set Synchronization of Audio and Video Streams

### **syncToAudio**

*boolean syncToAudio(int nPort, int nSyncToAudio)***Description:**

Set the synchronization of audio and video streams.

**Parameters:****nPort**

[in] Playing port number

**nSyncToAudio**

[in] Whether to open the synchronization of audio and video streams. 0 is disables, 1 is enabled.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

## 5.4.19 Set Window Transparency **setWindowTransparency**

*boolean setWindowTransparency(int nPort, float fTrans)***Description:**

Set the window transparency.

**Parameters:****nPort**

[in] Playing port number

**fTrans**

[in] Transparency parameter, the default value is 0.5. Range: 0~1.0.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

## 5.4.20 Refresh Current Display Window **refreshPlay**

**API Definition:**

```
boolean refreshPlay(  
    int    nPort
```

)

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

Returns TRUE for success, and returns FALSE for failure.

**Remark:**

This API is mainly used for refreshing the data frame in both the software decoding and hardware decoding mode.

## 5.4.21 Get Online Positioning Offset **getPlayTimeOffset**

**API Definition**

```
int getPlayTimeOffset(  
    int      nPort,  
    int      nTime  
);
```

**Parameters**

**nPort**

[in] Playback port

**nTime**

[in] Located play time, unit: ms, range: (0, playing duration)

**Return Values**

Returns the value of offset for success, and returns -1 for failure.

**Remark**

- This API is mainly used to get the size offset of MP4 data at specific time and locate the file when playing MP4 stream remotely. Playing MP4 file is not supported.
- When playing MP4 stream, the following stream types are not supported:
  - MP4 stream encoded by MPEG4 is not supported
  - VCA search of MP4 stream is not supported
- See the application flow of this API below:
  - 1) Open the local file in stream mode and call this API to locate.
  - 2) Stop playing after locating and wait for the offset data (relocated data).
  - 3) Restart playing after the offset data is inputted.

**Note:** After calling this API, inputting data must be stopped. And then call `resetSourceBuffer` to clear the SDK cache. Finally, read the specific offset data again and write it to the SDK.
- User can specify a video time point to start palying, the SDK will return the size of offset data according to the specified time point. And then user resend the data to the SDK according to the data size.

## 5.4.22 Set Playing Mode **SetAudioTrackParam**

**API Definition:**

```
public boolean SetAudioTrackParam(  
    int nPort,  
    int nAudioTrackMode,  
    int nAudioSessionId  
);
```

**Parameters:**

|                        |                                                                                                                      |
|------------------------|----------------------------------------------------------------------------------------------------------------------|
| <i>nPort</i>           | [IN] Playing port number                                                                                             |
| <i>nAudioTrackMode</i> | [IN] Playing mode, currently only the STREAM_VOICE_CALL mode is supported, so this parameter can only be set to "0". |
| <i>nAudioSessionId</i> | [IN] Audio session ID, which should be obtained from the application layer. And its value should be larger than 0.   |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** This API is only used for audio and video AEC (Acoustic Echo Canceller) process during video intercom.

## 5.4.23 Set Absolute Timestamp for Video Playing

### **SetAbsTimeFlag**

**API Definition:**

```
public boolean setAbsTimeFlag(  
    int nPort,  
    int nFlag  
);
```

**Parameters:**

|              |                                                                                         |
|--------------|-----------------------------------------------------------------------------------------|
| <i>nPort</i> | Playing port number                                                                     |
| <i>nFlag</i> | Start or stop playing video by absolute timestamp, 1-start, 0-stop, other values-error. |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- This API is only available for EZVIZ products, and the absolute timestamp should be configured via EZVIZ APP.
- This API should be called after starting playing video, otherwise, error code "16" will be returned.

## 5.4.24 Set AGC Parameters **SetAGCParam**

**API Definition:**

```
public boolean setAGCParam(  
    int nPort,  
    int nEable,  
    int nAGCLevel
```

```
);
```

**Parameters:**

|                  |                                                                                                                                                                                                                                                                             |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>nPort</i>     | Playing port number                                                                                                                                                                                                                                                         |
| <i>nFlag</i>     | Enable or disable AGC, 1-enable, 0-disable, other values-error.                                                                                                                                                                                                             |
| <i>nAGCLevel</i> | AGC level, the value is between 0 and 30. Level 0: the data is not processed; Level 1 to level 30: the actual loudness is from -32 db to -3 db, that is, when <i>nAGCLevel</i> is 30, the loudness is -3 db, when <i>nAGCLevel</i> is 29, the loudness is -4 db, and so on. |

**Return Values:** Return *true* for success, and return *false* for failure.

## 5.4.25 Set Chunk Size for RTMP Stream **SetDemuxParam**

```
API Definition: public boolean setDemuxParam(
    int nPort,
    int nDemuxType,
    int nChunSize
);
```

**Parameters:**

|                   |                                                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------|
| <i>nPort</i>      | Playing port number                                                                                              |
| <i>nDemuxType</i> | Parsing type, which must be set to "1". Otherwise, error code "1" will be returned.                              |
| <i>nChunSize</i>  | Chunk size, whose value should be larger than or equal to 128 bytes, otherwise, error code "1" will be returned. |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- This API is only available for parsing RTMP stream, and it is required when parsing.
- RTMP parsing only supports video with H.264 encoding format and audio with AAC encoding format.
- RTMP parsing does not support two fields (top field and bottom field) of encoded stream.

## 5.4.26 Set ANR Algorithm Parameters **setANRParam**

```
API Definition: setANRParam(
    int nPort,
    int nEnable,
    int nANRLevel
)
```

**Parameters:**

|                  |                                                                                  |
|------------------|----------------------------------------------------------------------------------|
| <i>nPort</i>     | [IN] Playing port number.                                                        |
| <i>nEnable</i>   | Whether to enable multi-slice detection.                                         |
| <i>nANRLevel</i> | Noise reduction level, the value is between 0 and 5, and the default value is 3. |

**Return Values:** Return *true* for success, and return *false* for failure.

## 5.5 Get Playing or Decoding Information

### 5.5.1 Get File Time Duration **getFileTime**

```
long getFileTime(int nPort);
```

**Description:**

Get total file duration (unit: second).

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

The file's total duration in seconds

### 5.5.2 Get System Time **getSystemTime**

```
Boolean getSystemTime(int nPort, MPSystemTime stSystemTime);
```

**Description:**

Get total file duration (unit: second).

**Parameters:**

**nPort**

[in] Playing port number

**MPSystemTime**

[out] Structure PLAYM4\_SYSTEM\_TIME

PLAYM4\_SYSTEM\_TIME {

    DWORD dwYear;        //year

    DWORD dwMon;        //month

    DWORD dwDay;        //date

    DWORD dwHour;        //hour

    DWORD dwMin;        //minute

    DWORD dwSec;        //second

    DWORD dwMs;        //millisecond

**Returned Value:**

The file's total duration in seconds

### 5.5.3 Get File Frame Number **getFileTotalFrames**

```
int getFileTotalFrames (int nPort);
```

**Description:**

Get the total frame number of the file.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

The total frame count of the file.

### 5.5.4 Get Current Frame Rate **getCurrentFrameRate**

```
int getCurrentFrameRate (int nPort);
```

**Description:**

Get the current frame rate.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

The current frame rate

If the frame rate is lower than 1fps, this API will return 0.

### 5.5.5 Get Decoded Frame Number **getPlayedFrames**

```
int getPlayedFrames(int nPort);
```

**Description:**

Get the decoded frame number.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

The decoded frame number of the file

### 5.5.6 Get Original Image Size **getPictureSize**

```
boolean getPictureSize(int nPort, MPInteger stWidth, MPInteger stHeight);
```

**Description:**

Get the original image size of the video.

**Parameters:**

**nPort**

[in] Playing port number

**pWidth**

[out] original image width, i.e. for CIF/PAL video, the image width is 352

**pHeight**

[out] original image height, i.e. for CIF/PAL video, the image height is 288

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Note:**

This API gets the size of the latest decoded image. Therefore it should be called after playback starts to get the precise information. If the current resolution is 2CIF, the pic is 4CIF.

### 5.5.7 Set Expected Frame Rate **setExpectedFrameRate**

**API Definition:**

```
public boolean setExpectedFrameRate(  
    int      nPort,  
    float    fExpectedFrameRate,  
    int      nFlag  
);
```

**Parameters:** *nPort* [IN] Playing port number, the value is between 0 and 31.  
*fExpectedFrameRate* [IN] Expected frame rate  
*nFlag* [IN] Whether to take effect

**Return values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- This API must be called after playing the video.
- The expected frame rate will be invalid if it is higher than the actual frame rate, and error code will be returned if the expected frame rate is lower than 1.
- The expected frame rate is used to drop frame in case of bad rendering performance, which is totally different from the concept of getting current number of frames ([getPlayedFrames](#)).

## 5.6 Decoding and Control

### 5.6.1 Set Video Frame Decoding Type **setDecodeFrameType**

#### API Definition

```
boolean setDecodeFrameType(
    int  nPort,
    int  nFrameType
);
```

#### Parameters

##### **nPort**

[in] Playback channel No.

##### **nFrameType**

[in] Frame decoding types, see details below:

|                              |                                 |
|------------------------------|---------------------------------|
| #define DECODE_NORMAIL       | 0-Normal decoding               |
| #define DECODE_KEY_FRAME     | 1-Decode key frame only         |
| #define DECODE_NONE          | 2-Not decode video frame        |
| #define DECODE_VIDEO_HALF    | 3-Decode 1/2 of SVC stream only |
| #define DECODE_VIDEO_QUARTER | 4-Decode 1/4 of SVC stream only |
| #define DECODE_VIDEO_EIGHTH  | 5-Decode 1/8 of SVC stream only |
| #define DECODE_ALL_FRAME     | 6-Decode all (default)          |

#### Return Value

Returns true for success, and returns false for failure.

#### Remark

- Call this API before decoding.
- When the decoding type is 4, the frame of high resolution stream will not be extracted automatically during ×8 speed playback.
- For real-time stream, when the decoding type is set to 2, audio stuck may occur.

### 5.6.2 Set Skipping I Frame Decoding **setIFrameDecInterval**

#### API Definition:

```
boolean setIFrameDecInterval(
    int  nPort,
    int  nInterval
)
```

#### Parameters:

*nPort*                      Playing port number  
*nInterval*                  Set the number of I frame to skip.

**Return Values:** Return *true* for success, and return *false* for failure.

#### Remarks:

- This API is mainly used to set parameters for decoding I frames, and before calling it, you must call [setDecodeFrameType](#) to set the parameter *nFrameType* to 1 (only decode I frame), otherwise, error code 2 will be returned.

- Support SVC (Scaled Video Coding), and you can call [setDecodeFrameType](#) to set it.

### 5.6.3 Decoding Callback **setDecodeCB**

*boolean setDecodeCB(int nPort, PlayerDecodeCB decodeCB);*

**Description:**

Register a callback function for user-controllable display.

**Parameters:**

**nPort**

[in] Playing port number

**DecCBFun**

[in] Pointer of the decoder callback function; can't be set as NULL.

**Description of the Callback Function:**

**public void onDecode(int nPort, byte[] data, int nDataLen, int nWidth, int nHeight, int nFrameTime, int nDataType, int Reserved);**

**Parameters:**

**nPort**

Playing port number

**data**

Pointer of the decoded video/audio buffer

**nDataLen**

Size of pBuf

**nWidth**

Image width in pixels or sound track number

**nHeight**

Image height in pixels or audio sample rate

**nFrameTime**

Time stamp in milliseconds

**nDataType**

Received data type as the Macro Definition below

**Reserved**

Reserved

**Macro Definition:**

|                        |                                                                                                                                  |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>T_AUDIO16 = 101</b> | PCM Audio, sampling rate: 16 Khz, Mono, 16 bits                                                                                  |
| <b>T_AUDIO8 = 100</b>  | PCM Audio, sampling rate: 8 Khz, Mono, 16 bits                                                                                   |
| <b>T_RGB32 = 7</b>     | RGB 32 image, 4 bytes per pixel arranged in 'B-G-R-0...' similar as bitmap (starts from the bottom-left of the image) (Reserved) |
| <b>T_UYVY = 1</b>      | uyvy image, arranged in "U0-Y0-V0-Y1-U2-Y2-V2-Y3...." (starts from the bottom-left of the image) format (Reserved)               |
| <b>T_YV12 = 3</b>      | yv12 image arranged in "Y0-Y1-....." "V0-V1....." "U0-U1-....." format                                                           |

**Returned Value:**

true - API calling succeeds;

false - API calling fails.



**Returned Value:** Return *true* on success, and return *false* on failure.

## 5.6.5 Set Decoding Callback (with Global Time)

### setDecodeCBEx

**Function:** boolean setDecodeCBEx (  
     int *nPort*,  
     PlayerDecodeCBEx *decodeCBEx*  
 );

**Parameter:** *nPort*  
     [in] Playing channel No.  
*decodeCBEx*  
     [in] Decoding callback function..

**Callback Function:**

```
public void (PlayerDecodeCBEx onDecodeEx)(
int nPort,
byte[] data,
int nDataLen,
int nWidth,
int nHeight,
int nFrameTime,
int nDataType,
int Reserved,
int nYear,
int nMon,
int nDay,
int nHour,
int nMin,
int nSec,
int nMs
);
```

**Callback Function Parameters:**

|        |            |                                                        |
|--------|------------|--------------------------------------------------------|
| int    | nPort      | Playing port number                                    |
| byte[] | data       | Decoded video and audio data pointer                   |
| int    | nDataLen   | Length of decoded video and audio data pBuf            |
| int    | nWidth     | Image width/sound channel number                       |
| int    | nHeight    | Image height/sound sampling rate                       |
| int    | nFrameTime | Decoding time stamp                                    |
| int    | nDataType  | Frame types, see details in Macro Definition as below. |
| int    | Reserved   | Reserved                                               |
| int    | nYear      | Year                                                   |
| int    | nMon       | Month                                                  |
| int    | nDay       | Day                                                    |

|     |       |             |
|-----|-------|-------------|
| int | nHour | Hour        |
| int | nMin  | Minute      |
| int | nSec  | Second      |
| int | nMs   | Millisecond |

**Macro Definition:**

|           |        |                                                                                                                                                              |
|-----------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T_AUDIO16 | = 101; | Audio data, sampling rate is 16 kHz, monaural, each sampling point is indicated by 16 bits.                                                                  |
| T_AUDIO8  | = 100; | Audio data, sampling rate is 8 kHz, monaural, each sampling point is indicated by 16 bits.                                                                   |
| T_RGB32   | = 7;   | Video data. Each pixel contains 4 bytes, and its order is same with bitmap. Format: "B-G-R-0 ...", the first pixel is at the lower left corner of the image. |
| T_UYVY    | = 1;   | Video data, format: uyvy. Example: "U0-Y0-V0-Y1-U2-Y2-V2-Y3...", the first pixel is at the lower left corner of the image.                                   |
| T_YV12    | = 3;   | Video data, format: yv12. Order: "Y0-Y1-...", "V0-V1...", "U0-U1-...".                                                                                       |

**Returned Value:** Return *true* on success, and return *false* on failure.

- Remarks:**
- Set callback function, and replace the OSD information by the user. This function should be called before calling *play*, and it will be invalid when calling *stop*. The function should be set again for the next call of *play*.
  - Speed is not controlled during decoding a part of stream. The decoder will decode the other part of stream as long as the stream returned from callback function by the user.
  - The current decoded video data format is YV12, and audio data format is PCM.
  - After calling this callback function, no image will display.
  - The year, month, day, hour, minute, second and millisecond information obtained from callback function, are the global time parsed from the current data stream. The global time is inputted during DSP stream encoding.

**Remark:** Calling APIs with nPort parameter in this callback function may cause locked.

## 5.6.6 Set File End CallBack **setFileEndCB**

```
boolean setFileEndCB(int nPort, PlayerPlayEndCB playEndCB);
```

**Description:**

Set callback for decoding file end. This API should be called before openSteam() or openFile ().

**Parameters:****nPort**

[in] Playing port number

**playEndCB**

[in] Callback function described below

**Description of the Callback Function:**

```
public void onPlayEnd(int nPort);
```

**Parameters:**

**nPort***Playing port number***Returned Value:**

true - API calling succeeds;  
false - API calling fails.

**Remark:**

For the callback functions, it must call the API before the Openfile.

## 5.6.7 Set Decoding Key **setSecretKey**

```
boolean setSecretKey(int nPort, int nKeyType, byte[] pSecretKey, int nKeyLen);
```

**Description:**

If a secret key has been set for encryption purpose during the encoding process, then this API should be called before openSteam or openFile for decryption.

**Parameters:****nPort**

[in] Playing port number

**nKeyType**

[in] Secret key type

**pSecretKey**

[in] Secret key string

**nKeyLen**

[in] Key length (unit: bit)

**Returned Value:**

true - API calling succeeds;  
false - API calling fails.

**Remark:**

The interface does not support the Android platform.

## 5.6.8 Set Decoding Tolerance Level **setDecodeERC**

**API Definition:**

```
public boolean setDecodeERC  
(  
    int    nPort,  
    int    nERCLLevel  
);
```

**Parameters:**

|            |                                                                                                                                                                                                      |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| nPort      | Playing port number                                                                                                                                                                                  |
| nERCLLevel | Tolerance level: 1, 2, 3, 4. Higher level indicates higher tolerance capability, but lower the decoding performance. If the level is set to 0, it indicates that the tolerance function is disabled. |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- Setting tolerance level is not supported by hardware decoding, but if it is

configured, no error will be returned.

- By default, the tolerance function is enabled in Intra flash stream.
- If the decoding tolerance is disabled, you cannot get the stream starts from P frame, otherwise, snow image may appear.
- When the decoding tolerance is disabled, if you re-decode the I frame, and then input P frame for decoding, snow image may appear.

## 5.6.9 Set Number of Decoding Threads

### setDecodeThreadNumber

**API Definition:**

```
public boolean setDecodeThreadNumber(  
    int    nPort,  
    int    nThreadNumber  
);
```

**Parameters:** *nPort* [IN] Playing port number, the value is between 0 and 31.  
*nThreadNumber* [IN] Number of decoding threads, which is between 1 and 8.

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- Only the stream with H.264 or H.265 format (including smart264 and smart265) supports setting number of decoding threads.
- For hardware decoding, setting decoding thread is not supported.
- When the number of threads is changed during video playing, the image may stick due to different stream resolution.

## 5.7 Display Operation

### 5.7.1 Set or Add Display Region **setDisplayRegion**

**API Definition:**

```
boolean setDisplayRegion(  
    int      nPort,  
    int      nRegionNum,  
    MPRect   pSrcRect,  
    Surface  hDestWnd,  
    int      bEnable  
);
```

**Description:**

Set or add display regions, also applies to digital zoom.

**Parameters:**

**nPort**

[in] Playing port number

**nRegionNum**

[in] Display region number from 0 to (MAX\_DISPLAY\_WND-1). If nRegionNum is set as 0, it stands for setting of the main display window.

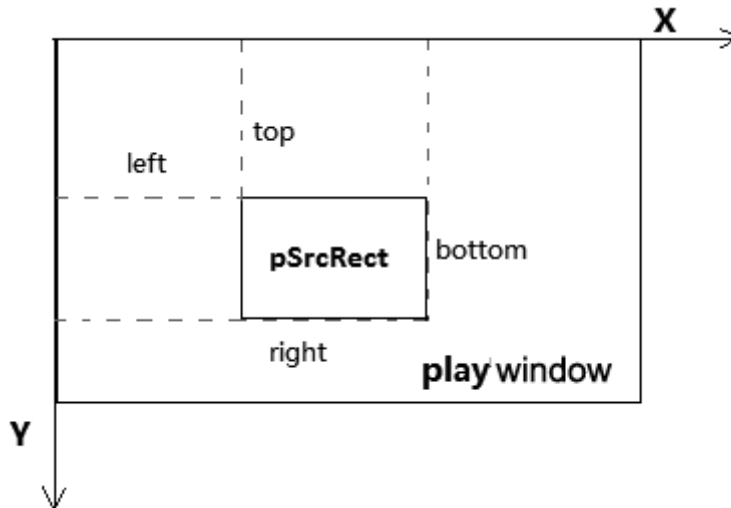
**MPRect pSrcRect**

[out] Set the region to be displayed (this region should be inside the original image), i.e. if the resolution of original image is 352\*288, the range of pSrcRect should not exceed (0, 0, 352, 288).

If pSrcRect is set as NULL, the whole image will be displayed.

The MPRect structure parameters description is shown below.

```
public static class MPRect  
{  
    public int left;      X axis coordinate on the upper-left corner of the area  
    public int right;     X axis coordinate on the lower-right corner of the area  
    public int top;       Y axis coordinate on the upper-left corner of the area  
    public int bottom;    Y axis coordinate on the lower-right corner of the area  
}
```

**hDestWnd**

[in] Set the display window. If the window hasn't been set, set hDestWnd as display window; If the window has already been set, and the display window is consistent with hDestWnd, neglect this parameter; If the window has already been set, and the display window is inconsistent with hDestWnd, change to hDestWnd dynamically.

**bEnable**

[in] Open/set or close the display region.

**Return Values:**

Returns TRUE for success, and returns FALSE for failure.

**Remark:**

- This API can be called only when the play is started, you can set or add the display area to realize the digital zoom.
- When the nRegionNum is 0, the settings of hDestWnd and bEnable will be ignored and the main window is under process. The value of digital zoom area (pSrcRect) must be larger than 0, and the value of right and bottom must be larger than 16. Otherwise, only display the original image.
- When in hardware decoding mode, the digital zoom function can only be realized in the original window.
- When realizing the digital zoom function, if returns failure, you can still call siwtchtohard to switch to the hardware decoding.

## 5.7.2 Set or Add Display Region **setDisplayRegionEx**

**API Definition:**

```
boolean setDisplayRegionEx(
    int          nPort,
    int          nRegionNum,
    MPRect       pSrcRect,
    SurfaceTexture texture,
    int          bEnable
);
```

**Parameters:**

nPort                  Playing channel No.

**nRegionNum**      Display region index: from 0 to (MAX\_DISPLAY\_WND-1)

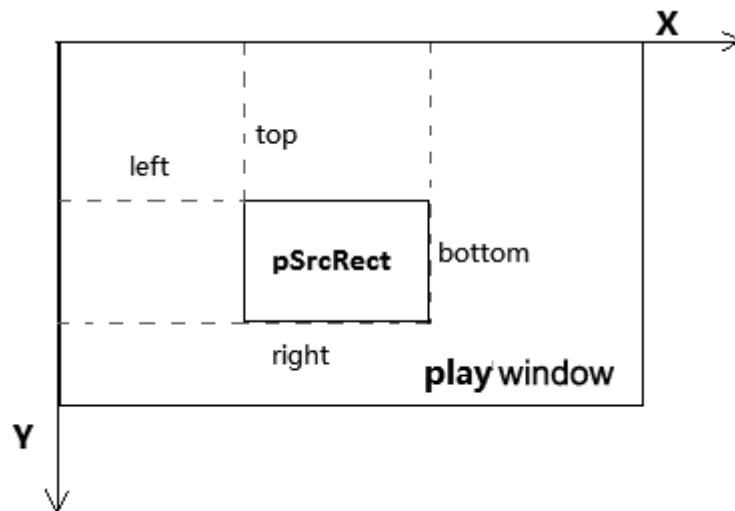
**pSrcRect**        Set the display region of original image. For example, if the resolution of original image is 352\*288, the available setting range of pSrcRect is (0, 0, 352, 288). If pSrcRect=NULL, the whole image will display.

**texture**           Set display window. If the window handle of hDestWnd is same with that of play API, the image in the selected region will directly display on the window, while source video will not display. If the hDestWnd has a new window handle, the image on the original window will not change, and the image in the selected region will display on the window which has the same handle with the region.

**bEnable**           Open (set) or close display region

MPRect Structure:

```
public static class MPRect
{
    public int left;        X axis coordinate on the upper-left corner of the area
    public int right;      X axis coordinate on the lower-right corner of the area
    public int top;        Y axis coordinate on the upper-left corner of the area
    public int bottom;    Y axis coordinate on the lower-right corner of the area
}
```



**Return Values:**

Returns TRUE for success, and returns FALSE for failure.

**Remark:**

- This API can be called only when the play is started, you can set or add the display area to realize the digital zoom.
- When the nRegionNum is 0, the settings of hDestWnd and bEnable will be ignored and the main window is under process. The value of digital zoom area (pSrcRect) must be larger than 0, and the value of right and bottom must be larger than 16. Otherwise, only display the original image.
- When in hardware decoding mode, the digital zoom function can only be realized in the original window.

- When realizing the digital zoom function, if returns failure, you can still call `siwtchtohard` to switch to the hardware decoding.

### 5.7.3 Set or Destroy Display Window **setVideoWindow**

**API Definition:**

```
boolean setVideoWindow(  
    int          nPort,  
    int          nRegionNum,  
    SurfaceHolder holder  
);
```

**Description:**

Set or destroy the display area.

**Parameters:**

**nPort**

[in] Playing port number

**nRegionNum**

[in] Display area No., the value is between 0 and (MAX\_DISPLAY\_WND-1).

**holder**

[in] Handle of display window

**Return Values:**

Returns TRUE for success, and returns FALSE for failure.

**Remark:**

- If you want to switch the window, you should call this API to set the original window as null, and then call related API to set a new display window.
- Call this API to create `SurfaceHolder` during the `SurfaceCreated` period, while call this API to destroy the `SurfaceHolder` during the `SurfaceDestroyed` period.
- The digital zoom will be disabled if this API is called.
- Destroying the original window is supported in hardware decoding mode, and after destroying, the image will keep at the current frame.

### 5.7.4 Set or Destroy Display Window **setVideoWindowEx**

**API Definition:**

```
boolean setVideoWindowEx(  
    int          nPort,  
    int          nRegionNum,  
    SurfaceTexture texture  
);
```

**Description:**

Set or destroy the display area.

**Parameters:**

nPort                      Playing port number

nRegionNum                Port index

texture                    Set handle of display window

**Return Values:**

Returns TRUE for success, and returns FALSE for failure.

- Remark:**
- If you want to switch the window, you should call this API to set the original window as null, and then call related API to set a new display window.
  - Call this API to create SurfaceHolder during the SurfaceCreated period, while call this API to destroy the SurfaceHolder during the SurfaceDestroyed period.
  - The digital zoom will be disabled if this API is called.
  - Destroying the original window is supported in hardware decoding mode, and after destroying, the image will keep at the current frame.

## 5.7.5 Set Synchronous Playback Group **setSyncGroup**

**API Definition:**

```
public boolean setSycGroup(  
    int    nPort,  
    int    nGroupIndex  
);
```

**Parameters:**

**nPort**

[in] Playing port number

**nGroupIndex**

[in] Synchronous playback group No., the value should be 0 to 4, hardware decoding is not supported.

**Return Values:**

Returns TRUE for success, and returns FALSE for failure.

**Remark:**

- You can call this API repeatedly to add the specific nPort to the synchronous playback group.
- Call setSycGroup before calling API to start playing.
- One nGroupIndex indicates one group, up to 16 channels can be added to each group. Otherwise, FALSE will be returned, and you must release the port (call freePort) to remove the channel from the group.
- Synchronous playback in file mode and stream mode are both supported.
- Global time is needed when in synchronous playback, so only Hikvision devices can be supported.
- When in hardware decoding mode, synchronous playback of multiple channels is also supported. To display one frame, multiple frames should be sent to the decoder, so at the beginning, there is no image.

## 5.7.6 Enable Feature Tracking Effect (for EZVIZ Client)

### **enableSuperEyeEffect**

**API Definition:** boolean enableSuperEyeEffect(

```
    int    nPort,  
    int    nRegionNum  
);
```

**Parameters:** *nPort* [IN] Playing port number.  
*nRegionNum* [IN] Display area No., which is between 0 and 5.

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- When calling this API, make sure you have opened files, file stream, or real-time stream in which private data of the type 0x1040 is contained.
- When feature tracking effect is enabled, private data of the type 0x1040 will have the priority over those of other types to be matched for each frame.
- The feature tracking effect is the same as that of digital zoom, which is performed according to parameters in private data of each frame.
- Mismatch between private data frame and video frame may occur during the fast forward playback.

### 5.7.7 Disable Feature Tracking Effect (for EZVIZ Client)

#### **disableSuperEyeEffect**

**API Definition:** `boolean disableSuperEyeEffect (`  
    `int nPort,`  
    `int nRegionNum`  
    `int nKeepEffect`  
`);`

**Parameters:** *nPort* [IN] Playing port number.  
*nRegionNum* [IN] Display area No., which is between 0 and 5.  
*nKeepEffect* [IN] Whether to keep the feature tracking effect. 1=yes;  
0=no, the image will turn back to the default effect.

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** When calling this API, make sure you have opened files, file stream, or real-time stream in which private data of the type 0x1040 is contained. And the feature tracking effect is enabled.

### 5.7.8 Get Parameters of the Current Display Area

#### **getCurrentRegionRect**

**API Definition:** `boolean getCurrentRegionRect (`  
    `int nPort,`  
    `int nRegionNum`  
    `MPRect stRect`  
`);`

**Parameters:** *nPort* [IN] Playing port number.

*nRegionNum* [IN] Display area No., which is between 0 and 5.  
*stRect* [OUT] Parameters of the current display area.

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** When calling this API, make sure you have opened files, file stream, or real-time stream.

## 5.7.9 Set Image Post-processing Parameters

### setImagePostProcessParameter

**API Definition:** `boolean setImagePostProcessParameter(  
     int nPort,  
     int nType,  
     float fValue  
 );`

**Parameters:** *nPort* Playing port number.  
*nType* Post-processing type, which is between 1 and 5.  
*fValue* Value of the post-processing type.

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- Post-processing types: 1-brightness (value range: [-1.0,1.0]), 2-chromaticity (value range: [0.0,1.0]), 3-saturation (value range: [-1.0,1.0]), 4-contrast (value range: [-1.0,1.0]), 5-sharpness (value range: [0.0,1.0]).
- The post-processing parameters will take effect only for the main image (the first image to be rendered).

## 5.8 Source Buffer Operation before Decoding

### 5.8.1 Get Remaining Data Size of Source Buffer

#### getSourceBufferRemain

`int getSourceBufferRemain(int nPort);`

**Description:**

Get the free space information of the source buffer (the *nPoolBufSize* of *openStream*).

**Parameters:**

**nPort**  
 [in] Playing port number

**Returned Value:**

Free space of the source buffer (unit: Byte).

## 5.8.2 Clear All Buffers **resetSourceBuf**

*boolean resetSourceBuf(int nPort);*

**Description:**

Clear the data in all the buffers.

**Parameters:**

**nPort**

[in] Playing port number

**nBufType**

[in] Buffer type

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

## 5.8.3 Clear Specified Buffer **resetBuffer**

*boolean resetBuffer(int nPort, int nBufType);*

**Description:**

Clear the data in a specified buffer.

**Parameters:**

**nPort**

[in] Playing port number

**nBufType**

[in] Buffer type

**BUF\_VIDEO\_SRC:** Video source buffer, valid for stream mode.

**BUF\_AUDIO\_SRC:** Audio source buffer, valid for video/audio separate stream mode.

**BUF\_VIDEO\_RENDER:** Video decode buffer.

**BUF\_AUDIO\_RENDER:** Audio decode buffer.

**Returned Value:**

true - API calling succeeds;

false - API calling fails

## 5.9 Playing Buffer Operation after Decoding

### 5.9.1 Set Frame Number in Playing Buffer **setDisplayBuf**

```
boolean setDisplayBuf(int nPort, int nNum);
```

**Description:**

Set the buffer size for playback (buffer of decoded images). This buffer is directly related with the fluency and real-time performance of playback. Larger buffer size usually means higher fluency yet inter time delay, and thus it is suggested to set larger buffer size for OpenFile mode (if the system memory is large enough). The default buffer size would be 10 (frames) for Open File mode, which is, about 0.6 sec under cases of 25fps; and 10 (frames) for Open Stream mode.

**Parameters:**

**nPort**

[in] Playing port number

**nNum**

[in] The number of frames to be buffered, range: from MIN\_DIS\_FRAMES to MAX\_DIS\_FRAMES

**MIN\_DIS\_FRAMES**

The minimum number of image frames to be buffered.

**MAX\_DIS\_FRAMES**

The maximum number of image frames to be buffered.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remark:**

This function should be called between openStream and play.

### 5.9.2 Get Frame Number in Playing Buffer **getDisplayBuf**

```
int getDisplayBuf(int nPort);
```

**Description:**

Get the buffer size (unit: frame number) of the playback buffer.

**Parameters:**

**nPort**

[in] Playing port number

**Returned Value:**

The maximum number of image frames to be buffered.

## 5.10 Source Buffer and Playing Buffer Operation

### 5.10.1 Get Specified Buffer Size **getBufferValue**

*long getBufferValue (int nPort, int nBufType);*

**Description:**

Get the data size of specified buffer.

**Parameters:**

**nPort**

[in] Playing port number

**nBufType**

[in] Buffer type

**BUF\_VIDEO\_SRC:** Video source buffer, valid for stream mode (unit: Byte)

**BUF\_AUDIO\_SRC:** Audio source buffer, valid for video/audio separate stream mode (unit: Byte)

**BUF\_VIDEO\_RENDER:** remained data for video decode buffer (unit: Frame)

**BUF\_AUDIO\_RENDER:** remained data for audio decode buffer (unit: Frame, every 40ms audio is divided as a frame)

**Returned Value:**

Current data length in the specified buffer.

**Remark:**

Get SDK buffer size (frame or byte). **BUF\_VIDEO\_RENDER** can be used for estimating network delay time.

## 5.11 Capture

### 5.11.1 Set Capture Callback Function **setDisplayCB**

*boolean setDisplayCB(int nPort, PlayerDisplayCB displayCB);*

**Description:**

Register a callback function for image capturing.

**Parameters:**

**nPort**

[in] Playing port number

**DisplayCBFun**

[in] Capturing image callback function

**Description of the Callback Function:**

```
public void onDisplay(int nPort, byte[] data, int nDataLen, int nWidth,  
int nHeight, int nFrameTime, int nDataType, int Reserved);
```

**Parameters:**

**nPort**

Playing port number

**data**

Pointer of the image capturing buffer

**nDataLen**

Size of the image capturing buffer

**nWidth**

Width of the image (unit: pixels)

**nHeight**

Height of the image (unit: pixels)

**nFrameTime**

Time stamp (unit: ms)

**nDataType**

Received data type: *T\_YV12*, *T\_RGB32*, *T\_UYVY*

**nReserved**

Reserved

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

- The callback should return ASAP, as the callback is triggered in clock thread and any time-consuming operation will affect the clock pulse and cause display problems. Users can stop the callback by setting DisplayCBFun as NULL.
- This API can be called at any time, and it will remain valid until application ends.
- Calling APIs with nPort parameter in this callback function may cause locked.

## 5.11.2 Capture Callback (with Global Time) **setDisplayCBEx**

**Function:**       boolean setDisplayCBEx (  
                  int    *nPort*,  
                  PlayerDisplayCBEx   *displayCBEx*  
                  );

**Parameter:**    *nPort*  
                  [in] Playing channel No.  
                  *displayCBEx*  
                  [in] Capture callback function, it can be NULL.

### **Callback Function:**

```
typedef void (PlayerDisplayCBEx onDisplayEx)(
int nPort,
byte[] data,
int nDataLen,
int nWidth,
int nHeight,
int nFrameTime,
int nDataType,
int Reserved,
int nYear,
int nMon,
int nDay,
int nHour,
int nMin,
int nSec,
int nMs);
```

### **Callback Function Parameters:**

|        |                   |                                           |
|--------|-------------------|-------------------------------------------|
| int    | <i>nPort</i>      | Channel No.                               |
| byte[] | <i>data</i>       | Returned image data pointer               |
| int    | <i>nDataLen</i>   | Returned image data size                  |
| int    | <i>nWidth</i>     | Image width, unit: pixel                  |
| int    | <i>nHeight</i>    | Image height                              |
| int    | <i>nFrameTime</i> | Time stamp information, unit: millisecond |
| int    | <i>nDataType</i>  | Data type: T_YV12, T_RGB32, T_UYVY        |
| int    | <i>Reserved</i>   | Reserved                                  |
| int    | <i>nYear</i>      | Year                                      |
| int    | <i>nMon</i>       | Month                                     |
| int    | <i>nDay</i>       | Day                                       |
| int    | <i>nHour</i>      | Hour                                      |
| int    | <i>nMin</i>       | Minute                                    |
| int    | <i>nSec</i>       | Second                                    |
| int    | <i>nMs</i>        | Millisecond                               |

**Returned Value:** Return *true* on success, and return *false* on failure.

**Remarks:**

- Set capture callback function. Set the callback pointer displayCBEx as NULL to stop the callback. The configured callback function will keep valid until closeStream is called. This function can be called at any time.
- If the callback is triggered in the clock thread, operations with long time consumption cannot be executed. Otherwise, the clock pulses will be disrupted and the display will be affected.
- The year, month, day, hour, minute, second and millisecond information obtained from callback function, are the global time parsed from the current data stream. The global time is inputted during DSP stream encoding.

**Remark:** Calling APIs with nPort parameter in this callback function may cause locked.

### 5.11.3 Capture BMP Picture **getBMP**

#### API Definition

```
boolean    getBMP(
    int          nPort,
    byte         pBmp,
    int          nBufSize,
    MPIInteger    pBmpSize
);
```

#### Parameters

##### **nPort**

[in] Playback channel No.

##### **pBitmap**

[in] Buffer allocated by users for saving BMP picture. The storage space (sizeof(BITMAPFILEHEADER)+sizeof(BITMAPINFOHEADER)+w\*h\*4, w and h is the width and height of the picture) should be larger than or equal to the BMP picture size.

##### **nBufSize**

[in] Applied buffer size

##### **pBmpSize**

[out] Captured BMP picture size

#### Return Values

Return true for success, and return false for failure.

#### Remark

- After enabling hardware decoding, the buffer allocated for saving BMP picture relates to the window resolution instead of the actual stream resolution, so the buffer size cannot be calculated by sizeof(BITMAPFILEHEADER)+sizeof(BITMAPINFOHEADER)+w\*h\*4. You can input null via pBitmap to get the required buffer size.
- For video with 2CIF resolution, the resolution of captured picture is 4CIF.

## 5.11.4 Capture BMP Picture with Private Information

### getBmpEx

|                       |                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API Definition</b> | <pre>public boolean getBmpEx (     int      nPort,     byte     pBitMap,     int      nBufferSize,     MPInteger nBitMapSize );</pre>                                                                                                                                                                                                                                                                |
| <b>Parameters</b>     | <p><b>nPort</b> Playback channel No.</p> <p><b>pBitMap</b> Buffer allocated by users for saving BMP picture. The storage space (sizeof(BITMAPFILEHEADER)+sizeof(BITMAPINFOHEADER)+w*h*4, w and h is the width and height of the picture) should be larger than or equal to the BMP picture size.</p> <p><b>nBufSize</b> Applied buffer size</p> <p><b>nBitMapSize</b> Captured BMP picture size.</p> |
| <b>Return Values</b>  | Returns <i>true</i> on success, and returns <i>false</i> on failure.                                                                                                                                                                                                                                                                                                                                 |
| <b>Remark</b>         | <ul style="list-style-type: none"> <li>● The cache allocated by BitMap should be larger than or equal to nBufferSize.</li> <li>● The size of picture captured via getbmpex relates to the window resolution instead of stream resolution.</li> <li>● Capturing picture with private information during hardware decoding is not supported.</li> </ul>                                                |

## 5.11.5 JPEG Capture **getJPEG**

```
boolean getJPEG(int nPort, byte[] pJpeg, int nBufSize, MPInteger pJpegSize);
```

### Description:

Capture image in JPEG format

### Parameters:

#### **nPort**

[in] Playing port number

#### **pJpeg**

[in] Address assigned by users for storing JPEG image, the file size should be no less than the JPEG file size, suggested value:  $w * h * \frac{3}{2}$ , in which w and h stand for the width and height of the image.

#### **nBufSize**

[in] assigned buffer size

#### **pBmpSize**

[out] Actual Jpeg image size captured.

### Returned Value:

true - API calling succeeds;

false - API calling fails.

**Remark:**

If the image width or height is not a multiple of 16, the captured image resolution will be automatically cropped to multiple of 16. E.g. original image of 176\*120 will be cropped to 176\*112.

### 5.11.6 Set Resolution to Capture BMP Picture

#### getBMPFixPixelEx

**API Definition:**    public boolean getBMPFixPixelEx(  
                                int                    *nPort*,  
                                byte[]                *pBMP*,  
                                int                    *nBufSize*,  
                                int                    *nWidth*,  
                                int                    *nHeight*,  
                                MPIInteger            *pBmpSize*  
                                );

**Parameters:**    *nPort*                    [IN] Playing port number, which is between 0 and 31.  
                                *pBMP*                    [IN] Data input buffer  
                                *nBufSize*                [IN] Size of data input buffer  
                                *nWidth*                [IN] Resolution width  
                                *nHeight*                [IN] Resolution height  
                                *pBmpSize*                [OUT] Actual size of BMP picture

**Return Values:**    Return *true* for success, and return *false* for failure.

**Remarks:**        ●    The minimum resolution is 16\*16 and the maximum resolution is 4096\*4096.  
                                ●    The actual width of captured picture will be automatically adjusted to a multiple of 4, which is the closest value smaller than or equal to the specified width.  
                                ●    If the input buffer is not configured, you can call this API to get the buffer size, and the size must be larger than or equal to the size of the picture to be captured.

### 5.11.7 Set Resolution to Capture Picture in Fisheye Mode

#### fecCaptureFixPixel

**API Definition:**    public boolean fecCaptureFixPixel(  
                                int                    *nPort*,  
                                int                    *nSubPort*,  
                                int                    *nType*,  
                                byte[]                *pBMP*,  
                                int                    *nBufSize*,  
                                int                    *nWidth*,  
                                int                    *nHeight*

```
);
```

**Parameters:**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>nPort</i>    | [IN] Playing port number, which is between 0 and 31.    |
| <i>nSubPort</i> | [IN] Fisheye sub port number, which is between 2 and 5. |
| <i>nType</i>    | [IN] Captured picture type, which can only be set to 1. |
| <i>pBMP</i>     | [IN] Data input buffer                                  |
| <i>nBufSize</i> | [IN] Size of data input buffer                          |
| <i>nWidth</i>   | [IN] Resolution width                                   |
| <i>nHeight</i>  | [IN] Resolution height                                  |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- The minimum resolution is 16\*16 and the maximum resolution is 4096\*4096.
- If the input buffer is not configured, you can get the buffer size via the API [fecGetCapPicSizeFixPixel](#), and the size must be larger than or equal to the size of picture to be captured.

### 5.11.8 Get Size of Picture Captured with Specified Resolution in Fisheye Mode **fecGetCapPicSizeFixPixel**

**API Definition:**

```
public boolean fecGetCapPicSizeFixPixel(
    int      nPort,
    int      nSubPort,
    int      nCapwidth,
    int      nCapHeight,
    MPInteger pBmpSize
);
```

**Parameters:**

|                   |                                  |
|-------------------|----------------------------------|
| <i>nPort</i>      | [IN] Playing port number         |
| <i>nSubPort</i>   | [IN] Fisheye sub port number     |
| <i>nCapwidth</i>  | [IN] Captured picture width      |
| <i>nCapHeight</i> | [IN] Captured picture height     |
| <i>pBmpSize</i>   | [OUT] Actual size of BMP picture |

**Return Values:** Return *true* for success, and return *false* for failure.

### 5.11.9 Capture JPEG Picture in Hardware Decoding Mode **getHDJPEG**

**API Definition:**

```
public boolean getHDJPEG(
    int      nPort,
    byte[]   pJpeg,
    int      nBufSize,
    int      nJPEGQuality,
    int      nWidth,
```

```
    int          nHeight,  
    MPIInteger   pJpegSize  
);
```

**Parameters:**

|                     |                                          |
|---------------------|------------------------------------------|
| <i>nPort</i>        | [IN] Playing port number                 |
| <i>pJpeg</i>        | [IN] Data buffer for saving JPEG picture |
| <i>nBufSize</i>     | [IN] Buffer size                         |
| <i>nJPEGQuality</i> | [IN] Picture quality                     |
| <i>nWidth</i>       | [IN] Input width                         |
| <i>nHeight</i>      | [IN] Input height                        |
| <i>pJpegSize</i>    | [OUT] Actual size of JPEG picture        |

**Return Values:** Return *True* for success, and return *False* for failure.

**Remarks:**

- The input buffer size should be larger than the value of  $1.5width*height+1024$ .
- The minimum resolution is 32\*32 and the maximum resolution is 4096\*4096.
- The JPEG picture of 2CIF captured in hardware decoding mode will be cropped into 4CIF.

## 5.12 Hardware Decoding

### 5.12.1 Set Hardware Decoding Priority **setHardDecode**

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API Definition</b> | <pre>boolean setHardDecode(     int      nPort,     int      bEnable );</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Parameters</b>     | <p>nPort      Playing channel No.</p> <p>bEnable    Open (set) or close the priority settings of hardware decoding: 0- Disable, 1- Enable</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Return Values</b>  | Returns TRUE for success, and returns FALSE for failure.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Remark:</b>        | <ul style="list-style-type: none"> <li>● After setting the priority of hardware decoding, the stream supports hardware decoding will be decoded first, and the other stream will be decoded by software decoding.</li> <li>● This API should be called after calling API to open stream (openStream) and before calling API to start playing (play).</li> <li>● After enabling hardware decoding, you must call setHardDecode again to disable hardware decoding, otherwise, you cannot enable the hardware decoding later.</li> <li>● During hardware decoding, when the frame rate is too low (<math>\leq 10</math>), resolution is too low (<math>&lt; 704 \times 576</math>), and the frame interval is too large (e.g., 100), the audio will be played earlier than the video for 1 to 4 seconds, so we recommend to switch to software decoding.</li> <li>● When synchronous playback is enabled during hardware decoding, if you switch the decoding mode to software decoding, the synchronous playback will not be effected.</li> <li>● After enabling hardware decoding, the following APIs are not supported: <ul style="list-style-type: none"> <li>➢ 2nd generation fisheye camera (setFECDisplayEffect, setFECDisplayParam, getFECDisplayParam);</li> <li>➢ Picture vertical flip (verticalFlip);</li> <li>➢ Decoding callback (setDecodeCB, setDecodeCBEx);</li> <li>➢ IVS private data callback (setIVSDrawFunCallback);</li> <li>➢ Reverse playback (reversePlay);</li> <li>➢ Capture (getJPEG);</li> <li>➢ Hardware decoding will directly be switched to software decoding in low frame rate (<math>\leq 1.0f</math>);</li> <li>➢ Hardware decoding will be directly switched to software decoding in low resolution (<math>W &lt; 200 \ \&amp;\&amp; \ H &lt; 200</math>);</li> <li>➢ Hardware decoding will be directly switched to software decoding in SCV mark stream;</li> <li>➢ Hardware decoding will be directly switched to software decoding in</li> </ul> </li> </ul> |

interlace stream;

- Hardware decoding will be switched to software decoding when decoding exception occurs.
- When display callback function and IVS private data callback function are called during software decoding, if you switch the decoding mode to hardware decoding, the two callback function will be disabled, the user should call them again.
- In the following situations, even if the priority of hardware decoding is configured, software decoding will still be adopted directly:
  - Stream with low frame rate ( $fRate \leq 1.0f$ )
  - Stream with low resolution ( $W < 200 \& \& H < 200$ )
  - Field encoded stream
  - SVC marked stream with multiple reference frames

## 5.12.2 Set Max. Hardware Decoding Channel Number

### setMaxHardDecodePort

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                          |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>API Definition:</b> | public boolean setMaxHardDecodePort(int nCount)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                          |
| <b>Parameters:</b>     | int nCount                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Maximum hardware decoding channel number |
| <b>Return Values:</b>  | Return true on success, or return false on failure.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                          |
| <b>Remarks:</b>        | <ul style="list-style-type: none"> <li>● This API should be called before enabling hardware decoding (setHardDecode).</li> <li>● Up to 8 channels' hardware decoding are allowed to be enabled, but the value of <i>nCount</i> can be set from 0 to 16. If the value is larger than 8, it will be automatically set to 8.</li> <li>● Hardware decoding is only available for system of version 4.1 or later, and it only supports the data encoded by H.264, H.265, MPEG4. Functions, such as digital zoom, capture, fisheye dewarping, reverse play, image vertical flipping, private data display and synchronous playback are not supported.</li> <li>● Set the maximum hardware decoding channel number, the default value is 4. No limit for calling switchToHard, e.g., if the current live view channel number is 3, while two channels is in hardware decoding (the configured maximum channel number is 2), the other software decoding channel can call switchToHard to switch to hardware decoding mode.</li> </ul> |                                          |

## 5.12.3 Get Current Decoding Mode **getDecoderType**

**API Definition:**

```
public int getDecoderType(
    int nPort
);
```

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameters:</b>    | nPort                      Playing channel No.                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Return Values:</b> | For hardware decoding, returns 1, for software decoding, returns 0.                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Remark:</b>        | <ul style="list-style-type: none"><li>● Call this API after the stream is playing normally, otherwise, the returned decoding type is invalid.</li><li>● Hardware decoding is only supported by the system with the version 4.1 or above, and it only supports the data encoded by H.264, H.265, and MPEG4.</li><li>● When the hardware decoding is enabled, only the functions of digital zoom in original window, BMP capture (including fisheye capture), and fisheye dewarping are supported.</li></ul> |

### 5.12.4 Switch to Software Decoding **switchToSoft**

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API Definition:</b> | <pre>boolean switchToSoft(<br/>    int      nPort<br/>);</pre>                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Description:</b>    | Manually switch the hardware decoding to software decoding.                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Parameters:</b>     | nPort                      Playing channel No.                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Return Values:</b>  | Returns TRUE for success, and returns FALSE for failure.                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Remark:</b>         | <ul style="list-style-type: none"><li>● If the digital zoom is enabled, error code 2 will be returned after calling this API.</li><li>● After calling this API, you should call related API (play) to start playing, otherwise, the software decoding may be disabled.</li><li>● If you directly call switchToHard instead of calling API (play) to start playing after calling this API, sequence error will occur.</li></ul> |

### 5.12.5 Switch to Hardware Decoding **switchToHard**

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API Definition:</b> | <pre>boolean switchToHard(<br/>    int      nPort<br/>);</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Parameters:</b>     | nPort                      Playback channel No.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Returned Value:</b> | Returns TRUE for success, and returns FALSE for failure.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Remark:</b>         | <ul style="list-style-type: none"><li>● If it is already the hardware decoding, error code 2 will be returned after calling this API.</li><li>● Before calling this API, you should destroy the decoder and wait for I frame until the video restarts to play after 2 to 3 seconds.</li><li>● By default, if the stream frame rate is smaller than 1, it will be switched to software decoding automatically.</li><li>● The following situations are not supported by this API:<ul style="list-style-type: none"><li>➢ The encoding types (except standard H264 (including Smart 264), standard H265 (including Smart 265), and MPEG4) are not supported by hardware decoding.</li><li>➢ Stream automatically switched to software decoding, including field</li></ul></li></ul> |

encoding, SVC marked stream, stream with low frame rate (width<200 and height<200).

- Set synchronous playback of multiple channels
- Enable digital zoom
- Enable 3rd generation fisheye mode

### 5.12.6 Set Whether to Reset Hardware Decoding when Exception Occurs **setResetHardDecodeFlag**

**API Definition:** `setResetHardDecodeFlag(  
    int    nPort,  
    bool   bResetFlag  
)`

**Parameters:** *nPort*                      Playing port number.  
*bResetFlag*                      Whether to reset hardware decoding when exception occurs.

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** This API should be called after [setHardDecode](#) and before playing the stream; otherwise, hardware decoding will be directly switched to software decoding when exception occurs.

## 5.13 Pre-recording

### 5.13.1 Set Pre-recording Data Callback **setPreRecordCallBack**

*boolean setPreRecordCallBack(int nPort, PlayerPreRecordCB PreRecordCB);*

**Parameters:**

**nPort**

[in] Playing port number

**PreRecordCB**

[in] Pre-recording callback function.

**pUser**

[in] User pointer.

**Data**

[in] audio and video data

**nDataLen**

[in] data length

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

- Call this API after the first frame is displayed normally.
- Calling API with nPort parameter in the callback function may cause locked.
- Pre-record function is to transform network stream to PS package, and then return to user by the callback function. It is convenient for users to view the stored file.
- The API setPreRecordCallBack should be called after the first I frame is displayed.
- The window handle cannot be *NULL* and it should be valid.

**Note:**

- In pre-record callback, the data should be processed timely. Executing IO operation for each callback is not recommended. The data should be cached and write to target file when the data is accumulated to a certain size (e.g., 2MB). If the data is not processed timely, or executing time-consuming IO operation frequently, the video playing speed and the VCA may mismatch, and thus the risk of frame drop of the video file may arise.
- Executing I/O operation too slow will affect the real-time of the video.
- The pre-record API should be set after setting the pre-recording status. Otherwise, return error code 2.

**Sample Code:**

```
int setPreRecordCallBack(int nPort, PlayerPreRecordCB);
```

```
//Disclaim prerecord callback function
```

```
void onPreRecordCBfun(int nPort,byte[] buffer,int nLen);
```

```
setPreRecordCallBack(IPort, PreRecordCBfun,NULL);
```

```

switch (dwDataType)
{
case NET_DVR_SYSHEAD:
    bFlag =getPort(&IPort);
    bFlag =setStreamOpenMode(IPort,1);
    bFlag =openStream(IPort,pBuffer,dwBufSize,4*1024*1024);

    bFlag =setPreRecordFlag(IPort,1);
    bFlag =setDecodeCB(IPort,DecCBFun);

    bFlag =play(IPort,pDlg->m_ctrlVideoPic.m_hWnd);
    break;
case NET_DVR_STREAMDATA:

    bFlag =inputData(IPort,pBuffer,dwBufSize);

    if ((g_bFlag== 0)&&(g_bDecFlag == 1))
    {
        Sleep(100);
        OutputDebugString("zytest:: set dec callback success !");
        g_bFlag =setPreRecordCallBack(IPort,fPreRecordCBfun,pUser);
        if (g_bFlag == 0)
        {
            unsigned int dwErr =GetLastError(IPort);
        }
        else
        {
            OutputDebugString("zytest:: set pre record callback success !");
        }
    }
    break;
}
}

```

### 5.13.2 Set Pre-recording Data Callback **setPreRecordCallBackEx**

**Function:** boolean setPreRecordCallBackEx(int nPort, PlayerPreRecordCBEx PreRecordCBEx)

**Parameters:** int nPort Playing channel No.  
 PlayerPreRecordCBEx PreRecordCBEx Data callback function

PlayerPreRecordCBEx Callback Function:  
 public void onPreRecordEx(int nPort, byte[]  
 data, int nDataLen, int nYear, int nMon, int

```
nDay, int nHour, int nMin, int nSec, int nMs,
int nType, int nFrameNum, int nStampTime);
```

Callback Function Parameters:

|        |            |                      |
|--------|------------|----------------------|
| int    | nPort      | Playing channel No.  |
| byte[] | data       | Video and audio data |
| int    | nDataLen   | Data length          |
| int    | nYear,     | Year                 |
| int    | nMon       | Month                |
| int    | nDay       | Day                  |
| int    | nHour      | Hour                 |
| int    | nMin       | Minute               |
| int    | nSec       | Second               |
| int    | nMs        | Millisecond          |
| int    | nType      | Frame type           |
| int    | nFrameNum  | Frame No.            |
| int    | nStampTime | Time stamp           |

**Returned Value:** Return 1 on success, and return 0 on failure.

**Remarks:**

- Call this API after the first frame is displayed normally.
- Calling API with nPort parameter in the callback function may cause locked.
- Pre-record function is to transform network stream to PS package, and then return to user by the callback function. It is convenient for users to view the stored file.
- The API setPreRecordCallBack should be called after the first I frame is displayed.
- The window handle cannot be *NULL* and it should be valid.

**Notes:**

- In pre-record callback, the data should be processed timely. Executing IO operation for each callback is not recommended. The data should be cached and write to target file when the data is accumulated to a certain size (e.g., 2MB). If the data is not processed timely, or executing time-consuming IO operation frequently, the video playing speed and the VCA may mismatch, and thus the risk of frame drop of the video file may arise.
- Executing I/O operation too slow will affect the real-time of the video.
- The pre-record API should be set after setting the pre-recording status. Otherwise, return error code 2.

### 5.13.3 Set Pre-recording Status **SetPreRecordFlag**

```
boolean setPreRecordFlag(int nPort, int bFlag)
```

**Parameters:**

**nPort**

[in] Playing port number

**bFlag**

[in] Set pre-recording status: 1- Enable, 0-Disable

**Returned Value:**

Return 1 on success, and return 0 on failure.

**Remarks:**

- Call this API after the first frame is displayed normally.
- Pre-record function is to transform network stream to PS package, and then return to user by the callback function. It is convenient for users to view the stored file.

**Note:**

2-second or 3-second delay may exist because of the cache.

## 5.14 Fisheye

### 5.14.1 Set Image Correction **setImageCorrection**

*boolean setImageCorrection(int nPort, int bEnable);*

**Description:**

Set to enable the wide-angle image correction or not.

**Parameters:**

**nPort**

[in] Playing port number

**bEnable**

[in] Open/set or close the image correction: 0- close, 1- open

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remark:**

120° image correction

### 5.14.2 Set Fisheye Dewarping Type **setFECDisplayEffect**

**API Definition**

```
boolean setFECDisplayEffect(  
    int                nPort,  
    int                nRegionNum,  
    MPVR_DISPLAY_EFFECT enDisplayEffect  
);
```

**Parameters**

**nPort**

[in] Playback channel No.

**nRegionNum**

[in] Window No.

**enDisplayEffect**

[in] Fisheye dewarping types, see details below:

VR\_ET\_NULL = 0, //No dewarping

VR\_ET\_FISH\_PTZ\_CEILING = 1, //Applicable to ceiling mounting

VR\_ET\_FISH\_PTZ\_FLOOR = 2, //Applicable to ground mounting

VR\_ET\_FISH\_PTZ\_WALL = 3, //Applicable to wall mounting

VR\_ET\_FISH\_PANORAMA\_CEILING360 = 4, //Applicable to ceiling mounting 1P

VR\_ET\_FISH\_PANORAMA\_CEILING180 = 5, //Applicable to ceiling mounting 2P

VR\_ET\_FISH\_PANORAMA\_FLOOR360 = 6, //Applicable to ground mounting 1P

VR\_ET\_FISH\_PANORAMA\_FLOOR180 = 7, //Applicable to ground mounting 2P

VR\_ET\_FISH\_LATITUDE\_WALL = 8, //Applicable to wall mounting (180° panorama)

#### Return Values

Returns true for success, and returns false for failure.

#### Remark

- Before calling this API, the video should be playing.
- After fisheye view dewarping, the image will restore to normal image.
- You can call setDisplayRegion and setVideoWindow to set a new window to start fisheye dewarping. The fisheye dewarping always works for original image.
- The value of enDisplayEffect is between 0 and 8. If this parameter is set to 9, error 16 will be returned. For other values, API returns 0, and there is no dewarping effect.

### 5.14.3 Get Fisheye Dewarping Parameter **getFECDisplayParam**

```
boolean getFECDisplayParam(int nPort, int nRegionNum, MPVR_FISH_PARAM stFishParam);
```

#### Parameters:

##### nPort

[in] Playing port number

##### nRegionNum

[in] Display region number.

##### stFishParam

[in] Fisheye correction parameter.

```
public static class MPVR_FISH_PARAM{
    public float  xLeft;      //The X coordinate on the far left (min).
    public float  xRight;     //The X coordinate on the far right (max).
    public float  yTop;       //The Y coordinate on the top (min).
    public float  yBottom;    //The Y coordinate on the bottom (max).
    public float  angle;      //180°correction central angle.
    public float  zoom;       //PTZ correction: zoom
    public float  PTZX;       //X coordinate of PTZ correction center.
    public float  PTZY;       //Y coordinate of PTZ correction center.
}
```

#### Return:

true - API calling succeeds;

false - API calling fails.

#### Description:

This API should be called after “play”

### 5.14.4 Set Fisheye Dewarping Parameter **setFECDisplayParam**

```
boolean setFECDisplayParam(int nPort, int nRegionNum, MPVR_FISH_PARAM stFishParam);
```

#### Description:

Set the fisheye correction parameter.

#### Parameters:

**nPort**

[in] Playing port number

**nRegionNum**

[in] Display region number.

**stFishParam**

[in] Fisheye correction parameter.

```

public static class MPVR_FISH_PARAM{
    public float xLeft;    //The X coordinate on the far left (min). Range: [0.0,1.0].
    public float xRight;  //The X coordinate on the far right (max). Range: [0.0,1.0].
    public float yTop;    //The Y coordinate on the top (min). Range: [0.0,1.0].
    public float yBottom; //The Y coordinate on the top (min). Range: [0.0,1.0].
    public float angle;   //180°correction central angle. Range: [0.0,360.0].
    public float zoom;    //PTZ correction: zoom. Range: (0.0,1.0).
    public float PTZX;    //X coordinate of PTZ correction center. Range: [0.0,1.0]
    public float PTZY;    //Y coordinate of PTZ correction center. Range: [0.0,1.0]
}

```

**Return:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

- The parameters should meet the following conditions:
  - xLeft < xRight;**
  - yTop < yBottom;**
  - The PTZ correction center should be in the circle of radius 0.4 and centered in (0.5,0.5).
- To get fisheye effect in new display region you should establish corresponding new display region through the use of digital zooming.

### 5.14.5 Enable Fisheye Dewarping **enableFEC**

**API Definition:** `boolean enableFEC(  
 int nPort  
);`

**Parameters:** *nPort* Playing port number

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks**

- The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.

### 5.14.6 Disable Fisheye Dewarping **disableFEC**

`boolean disableFEC(int nPort);`

**Parameters:****nPort**

[in] Playing port number.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remark:**

The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.

## 5.14.7 Get Fisheye Dewarping Sub Port **getPortFEC**

**API Definition:**

```
boolean getPortFEC (  
    int    nPort,  
    int    nPlaceType,  
    int    nCorrectType  
);
```

**Parameters:**

nPort            Playing channel No.

nPlaceType      Mounting type

**Mounting Types:**

```
public static class FEC_PLACETYPE  
{  
    public static final int FEC_NULL    = 0;  // Unmounted  
    public static final int FEC_PLACE_WAL= 1; // Wall mounting  
    public static final int FEC_PLACE_FLOOR= 2; // Ground Mounting  
    public static final int FEC_PLACE_CEILING= 3; // Ceiling Mounting  
}
```

**Dewarping Modes:**

```
public static class FEC_CORRECTTYPE  
{  
    public static final int FEC_CORRECT_PTZ= 0;  
    //PTZ  
    public static final int FEC_CORRECT_180= 1;  
    //Dual-180° panorama  
    public static final int FEC_CORRECT_360= 2;  
    //360° panorama  
    public static final int FEC_CORRECT_LAT= 3;  
    //180° panorama  
    public static final int FEC_CORRECT_SEM= 4;  
    //360° panorama+hemispherical surface  
    public static final int FEC_CORRECT_CYC= 5;  
    //180° panorama+cylindrical surface  
    public static final int FEC_CORRECT_PLA= 6;  
    //360° panorama+planet extension  
    public static final int FEC_CORRECT_CYC_SPL=7;  
    //180° panorama+extended cylindrical surface  
    public static final int FEC_CORRECT_ARC_HORIZONTAL = 8;
```

```
//180° panorama+horizontal curved surface (wall mounting)
public static final int FEC_CORRECT_ARC_VERTICAL    = 9;
//180° panorama+vertical curved surface (wall mounting)
public static final int FEC_CORRECT_PANOSPHERE =10;
//360° panorama+spherical surface
}
```

**Return Values:** Returns sub port No. (which is between 2 and 5) for success, and returns FALSE for failure.

- Remark:**
- The 3rd generation fisheye camera supports 4-channel expansion, the sub port No. is ranging from 2 to 5.
  - The render and fisheye function of the PlayCtrl Library SDK depend on same rendering library. The ISubPort starts from 2.
  - The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.
  - One port only corresponds to one mounting type.
  - When in hardware decoding mode, fisheye dewarping is only supported in original window. If you want to enable PTZ dewarping of ground mounting type, you can call `getPortFEC(nPort, FEC_PLACE_FLOOR, FEC_CORRECT_PTZ)`. All returned fisheye sub ports are 0. To recover to original view, call `getPortFEC(nPort, FEC_PLACE_FLOOR, FEC_CORRECT_PTZ)` again.
  - Different mounting types correspond to different dewarping types, see details in the following table:

| Mounting Type     | Supported Dewarping Mode                                                                                         |
|-------------------|------------------------------------------------------------------------------------------------------------------|
| FEC_NULL          | No dewarping                                                                                                     |
| FEC_PLACE_WALL    | FEC_CORRECT_PTZ<br>FEC_CORRECT_360<br>FEC_CORRECT_LAT<br>FEC_CORRECT_ARC_HORIZONTAL<br>FEC_CORRECT_ARC_VERTICAL  |
| FEC_PLACE_FLOOR   | FEC_CORRECT_180<br>FEC_CORRECT_360<br>FEC_CORRECT_SEM<br>FEC_CORRECT_CYC<br>EC_CORRECT_PLA<br>EC_CORRECT_CYC_SPL |
| FEC_PLACE_CEILING | FEC_CORRECT_180<br>FEC_CORRECT_360<br>FEC_CORRECT_SEM<br>FEC_CORRECT_CYC<br>EC_CORRECT_PLA<br>EC_CORRECT_CYC_SPL |

### 5.14.8 Delete Fisheye Dewarping Sub Port **delPortFEC**

*boolean delPortFEC(int nPort, int nSubPort)*

**Parameters:**

**nPort**

[in] Playing port number.

**nSubPort**

[in] The deleted sub port number. Value range: [2~5]

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Description:**

The 3-generation fisheye supports 4-channel expansion, the value range is [2~5]

**Remark:**

The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.

### 5.14.9 Set Fisheye Dewarping Parameters **setParamFEC**

**API Definition:** `boolean setParamFEC(  
                   int                  nPort,  
                   int                  nSubPort,  
                   FESHEYE\_PARAM  stParam  
                   )`

**Parameters:**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>nPort</i>    | Playing port number                   |
| <i>nSubPort</i> | Fisheye port No.                      |
| <i>stParam</i>  | Fisheye dewarping parameter structure |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.
- 4-channel dewarping is supported by 3rd fisheye camera, and the fisheye port No. is between 2 and 5.
- The absolute values of parameters *fZoom* and *fWideScanOffset* in the structure [FESHEYE\\_PARAM](#) must be larger than or equal to 0.00001, otherwise, there is no effect.
- Flipping is not supported.
- In file stream mode, when the last frame is played, the image will not change even if the fisheye dewarping parameter is configured. This is because the current frame is not refreshed repeatedly as the performance consideration.
- This API is not available for 3D fisheye dewarping.

### 5.14.10 Get Fisheye Dewarping Parameters **getParamFEC**

**API Definition:** `boolean getParamFEC(  
                   int                  nPort,  
                   int                  nSubPort,  
                   FESHEYE\_PARAM  stParam  
                   )`

**Parameters:** *nPort*                   Playing port number  
*nSubPort*                   Fisheye port No.  
*stParam*                   Fisheye dewarping parameter structure

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.
- 4-channel dewarping is supported by 3rd fisheye camera, and the fisheye port No. is between 2 and 5.

### 5.14.11 Set Displaying Window **setWnd**

**API Definition:** `boolean setWnd(  
                   int                  nPort,  
                   int                  nSubPort,  
                   SurfaceHolder  holder  
                   )`

**Parameters:** *nPort*                   Playing port number  
*nSubPort*                   Fisheye port No.  
*holder*                   Set the display window handle.

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.
- 4-channel dewarping is supported by 3rd fisheye camera, and the fisheye port No. is between 2 and 5.
- When enabling PTZ dewarping for low frame rate file, the PTZ frame will be displayed until the next frame displayed.

**Example Code:**

```
boolean setWnd(int nPort, int nSubPort, SurfaceHolder holder); // enable the fisheye function
enableFEC (m_lPort));
getPortFEC (m_lPort,&m_nSubPort1,FEC_PLACE_CEILING,FEC_CORRECT_PTZ));

// upgrade the position and zooming parameters at same time
stFEPara1.nUpDateType = FEC_UPDATE_PTZPARAM | FEC_UPDATE_PTZZOOM;// update type
stFEPara1.stPTZParam.fPTZPositionX = 0.2; //coordinate of the PTZ playing central
stFEPara1.stPTZParam.fPTZPositionY = 0.3;
stFEPara1.fZoom = 0.1; //PTZ display range parameters
```

```

//stCycleParam applied to circle and can influence all of the sub ports.
setParamFEC (m_lPort,m_nSubPort1,&stFEPara1);

// sub port 2 mounted on ceiling, PTZ correction, get fisheye correction sub port
getPortFEC (m_lPort,&m_nSubPort2,FEC_PLACE_CEILING,FEC_CORRECT_PTZ);

// update location
stFEPara2.nUpDateType = FEC_UPDATE_PTZPARAM;
stFEPara2.stPTZParam.fPTZPositionX = 0.7;
stFEPara2.stPTZParam.fPTZPositionY = 0.8;
setParamFEC (m_lPort,m_nSubPort2,&stFEPara2);

//Zoom
stFEPara3.nUpDateType = FEC_UPDATE_PTZZOOM;
stFEPara3.fZoom          = 0.9; //(0,1)

setParamFEC (m_lPort,m_nSubPort2,&stFEPara3);

// update location
stFEPara3.nUpDateType = FEC_UPDATE_PTZPARAM;
stFEPara3.stPTZParam.fPTZPositionX = 0.7;
stFEPara3.stPTZParam.fPTZPositionY = 0.8;
stFEPara3.fZoom          = 0.9;
setParamFEC (m_lPort,m_nSubPort2,&stFEPara3))

//Zoom
stFEPara3.nUpDateType = FEC_UPDATE_PTZZOOM;
setParamFEC (m_lPort,m_nSubPort2,&stFEPara3);

setWnd (m_lPort,m_nSubPort1,hWnd1);

```

### 5.14.12 Set Displaying Window **setWndEx**

*boolean setWndEx(int nPort, int nSubPort, SurfaceTexture texture)*

**Parameters:**

**nPort**

[in] Playing port number.

**nSubPort**

[in] The sub port.

**texture**

[in] The displaying screen handle.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

The 3-generation fisheye supports 4 c hannel expansion, the sub port value range is [2~5]

**Note:**

The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time.

The API of version 7.3.2 and later is recommended.

### 5.14.13 Get Current Selected Fisheye Sub Port

#### getCurrentPTZPort

**API Definition:** `boolean getCurrentPTZPort(  
 int nPort,  
 int nPanorama,  
 float fPositionX,  
 float fPositionY  
)`

**Parameters:** *nPort*           Playing port number  
*nPanorama*   0-Operate original image, 1-Operate panorama image  
*fPositionX*   X-coordinate of cursor.  
*fPositionY*   Y-coordinate of cursor.

**Return Values:** Return *fisheye port No.* for success, and return *-1* for failure.

**Remarks:**

- The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.
- 4-channel fisheye expansions are supported by version 7.3.2, the fisheye port No. is between 2 to 5.

### 5.14.14 Set Current Selected Fisheye Sub Port

#### setCurrentPTZPort

`boolean setCurrentPTZPort(int nPort, int nSubPort)`

**Parameters:**

**nPort**

[in] Playing port number.

**nSubPort**

[in] Sub port number.

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

- The fisheye interface of Version 7.3.1 and previous and Version 7.3.2 and later can't be used at

the same time. Version 7.3.2 and later is recommended.

- Version 7.3.2 and later support 4 channel expansion, the value range of the sub port is [2~5].

This interface is newly-added in Version 7.3.2

### 5.14.15 Select by Drawing Rectangle or Irregular Frame

#### setPTZoutLineShowMode

*boolean setPTZoutLineShowMode(int nPort, int nShowmode)*

**Parameters:**

**nPort**

[in] Playing port number.

**nShowmode**

[in] The display mode.

public static class FEC\_SHOWMODE

{

public static final int FEC\_PTZ\_OUTLINE\_NULL = 0; //No display.

public static final int FEC\_PTZ\_OUTLINE\_RECT = 1; //Rectangle display (Not supported).

public static final int FEC\_PTZ\_OUTLINE\_RANGE= 2; //The real area display.

}

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

- The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.
- Version 7.3.2 and later support 4 channel expansion, the value range of the sub port is [2~5].

### 5.14.16 Get PTZ Coordinates of Current Sub Port

#### setPTZ2Window

*boolean setPTZ2Window(int nPort, int nSubPort, PTZ\_PARAM stPTZRefOrigin, PTZ\_PARAMM stPTZRefWindow, PTZ\_PARAM stPTZWindow, MPFloat stPosX, MPFloat stPosY)*

**Parameters:**

**nPort**

[in] Playing port number.

**nSubPort**

[in] The desired sub port number.

**stPTZRefOrigin**

[in] The current PTZ coordinates.

**stPTZRefWindow**

[in] The initial coordinate of touching control area by finger or mouse.

**stPTZWindow**

[in] The current coordinate of touching control area by finger or mouse.

**stPosX**

[in] The X coordinate of new calculated PTZ.

**stPosY**

[in] The Y coordinate of new calculated PTZ.

```
public static class PTZ_PARAM
```

```
{
    public float ptzPositionX; //The X coordinate of PTZ displaying central.
    public float ptzPositionY; // The Y coordinate of PTZ displaying central.
}
```

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

Get the current PTZ coordinates of the current sub port.

**Notes:**

- The fisheye interface of Version 7.3.1 and previous and Version 7.3.2 and later can't be used at the same time. Version 7.3.2 and later is recommended.
- Version 7.3.2 and later support 4 channel expansion, the value range of the sub port is [2~5].
- This interface is newly-added in version 7.3.2

## 5.14.17 Get PTZ Coordinates of Current Sub Port

### setFECDisplayPTZ

**Function:** boolean setFECDisplayPTZ(int nPort, int nSubPort, MPVR\_FISH\_PTZ stOriPtz, MPVR\_FISH\_PTZ stWndPtz, MPVR\_FISH\_PTZ stWnd, MPFloat stX, MPFloat stY)

|                   |                |          |                                           |
|-------------------|----------------|----------|-------------------------------------------|
| <b>Parameter:</b> | int            | nPort    | Playing channel No.                       |
|                   | int            | nSubPort | Set desired sub port No.                  |
|                   | PTZ_PARAM      |          | Current PTZ coordinate                    |
|                   | stPTZRefOrigin |          | Initial coordinate of control area        |
|                   | PTZ_PARAM      |          | Current coordinate of control area        |
|                   | stPTZRefWindow |          | Calculated X axis value of PTZ coordinate |
|                   | PTZ_PARAM      |          | Calculated Z axis value of PTZ coordinate |
|                   | stPTZWindow    |          |                                           |
|                   | MPFloat        | stPosX,  |                                           |
|                   | MPFloat        | stPosY   |                                           |

PTZ\_PARAM Structure PTZ central position, X axis

Description: PTZ central position, Y axis

```
public static class PTZ_PARAM
{
    public float ptzPositionX;
```

```
public float ptzPositionY;
}
```

**Returned Value:** Return *true* on success, and return *false* on failure.

**Remarks:**

- The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.
- Version 7.3.2 and later support 4 channel expansion, the value range of the sub port is [2~5].

## 5.14.18 Set Rotation Parameters of 3D Fisheye Dewarping

### setFish3DRotate

**API Definition:**

```
public boolean setFish3DRotate(
    int          nPort,
    int          nSubPort,
    SRTRANS_PARAM srTransParam
);
```

**Parameters:**

|                     |                                                                                                           |
|---------------------|-----------------------------------------------------------------------------------------------------------|
| <i>nPort</i>        | Playing port number                                                                                       |
| <i>nSubPort</i>     | Sub port number                                                                                           |
| <i>srTransParam</i> | Rotation parameters of 3D fisheye dewarping, see details in the structure <a href="#">SRTRANS_PARAM</a> . |

**Return Value:** Return *true* for success, and return *false* for failure.

**Remarks:** Value Ranges:

The values of parameter *fAxisX*, *fAxisY*, and *fValue* vary with the 3D fisheye dewarping modes, see details as the follows:

1) Value range of rotation around X axis:

- For asteroid-shaped view: from  $[-\pi/2, \pi/2]$ ;
- For dissected-cylindrical-surface-shaped view:  $[0, \pi/4]$ ;
- Rotation ranges of horizontal and vertical surface vary with the value of *fValue*:
  - When *fValue* is set to 2.0f, rotation range around X axis:
    - ✧ Horizontal-curved-surface-shaped view:  $[-\pi/18, \pi/18]$ ;
    - ✧ Vertical-curved-surface-shaped view:  $[-\pi/6, \pi/6]$ ;
  - When the value of *fValue* is not 2, the range of rotation value of X axis varies with the value of *fValue* to ensure that the whole cambered surface can be seen.
- For others: No limit.

2) Value range of rotation around Y axis:

- Rotation values of horizontal and vertical surface vary with the value of *fValue*:
  - When *fValue* is set to 2.0f, rotation range around Y axis:

- ✧ Horizontal-curved-surface-shaped view:  $[-\pi/3, \pi/3]$ ;
  - ✧ Vertical-curved-surface-shaped view:  $[-\pi/9, \pi/9]$ ;
  - When the value of fValue is not 2, the range of rotation value of Y axis varies with the value of fValue to ensure that the whole cambered surface can be seen.
    - For dissected-cylindrical-surface-shaped view: No limit for rotation value range, but the parameter value should be (-1.0, 1.0).
    - For others: No limit.
- 3) Zoom:
- For semisphere-shaped view:  $[-0.8, 900]$ ;
  - For cylindrical-surface-shaped view:  $[0, 900]$ ;
  - For asteroid-shaped view:  $[-0.3, 1.2]$ ;
  - For dissected-cylindrical-surface-shaped view: Not support;
  - For others: No limit.
- 4) When the value is out of range, the rotation value around X axis is limited for dissected-cylindrical-surface-shaped view (e.g., the range is from 0 to 1, setting -1 means limit to 0, setting 2 means limit to 1); for other effects, the error will be returned.
- 5) When rotating, the order of setting parameters is Y, X, and Value, so the previous rotation effect will not be affected by the error of the next one (e.g., if the rotation value around Y axis is correct and the X parameter is incorrect, the API will return error, but the image will still rotate according to the Y axis settings).
- 6) When setting fisheye 3D rotation, if the value is set to threshold value, returned values may vary because accuracy differs in different phones (e.g., if the value is set to -900.7 by the upper layer, the value in Samsung s6 will be -900.700012, while on other platforms the value may be -900.700000).
- 7) Default values of fisheye 3D rotation:
- For semisphere-shaped view:  
fAxisX =  $3\pi/2$   
fAxisY = 0.0f  
fValue = 3.0f
  - For cylindrical-surface-shaped:  
fAxisX =  $\pi/4$   
fAxisY = 0.0f  
fValue = 6.0f
  - For dissected-cylindrical-surface-shaped view:  
fAxisX = 0.0f  
fAxisY = 0.0f  
fValue = 0.0f
  - For asteroid-shaped view:  
fAxisX =  $\pi/2$   
fAxisY = 0.0f  
fValue = 1.0f
  - For vertical/horizontal-curved-surface-shaped view:

fAxisX = 0.0f

fAxisY = 0.0f

fValue = 2.0f

### 5.14.19 Get Cache Size for Fisheye Capture **getFishCapPicSize**

**Function:**           public boolean getFishCapPicSize(  
                          int     nPort,  
                          int     nSubPort,  
                          MPIInteger   nCapPicSize  
                          );

**Parameters:**     int                    nPort                   Playing channel No.  
                     int                    nSubPort               Set desired sub port No.  
                     [out]MPIInteger       nCapPicSize           Actual captured picture size

**Returned Value:** Return *true* on success, and return *false* on failure.

### 5.14.20 Capture in Fisheye Mode **getFishCapPic**

**Function:**           public boolean getFishCapPic (  
                          int     nPort,  
                          int     nSubPort,  
                          int     nType,  
                          byte[] pFishPic,  
                          int     nFishPicSize  
                          );

**Parameters:**     int                    nPort                   Playing channel No.  
                     int                    nSubPort               Set desired sub port No.  
                     int                    nType,                 Data type, only support data in YV12format.  
                     byte[]                pFishPic,               Picture buffer (applied by user)  
                     int                    nFishPicSize           Picture size

**Returned Value:** Return *true* on success, and return *false* on failure.

**Remarks:**       The value of nFishPicSize is provided by getFishCapPicSize, and the cache size allocated by pFishPic should be larger than or equal to nFishPicSize.

**Notes:**           Before calling this API to realize fisheye capture, call getFishCapPicSize to get the nFishPicSize.

### 5.14.21 Set Fisheye Capture Callback Function

#### **setFECDisplayCB**

**API Definition:**   public boolean setFECDisplayCB  
                      (

```

        int nPort,
        int nFishPort,
        PlayerCallBack.PlayerFECDisplayCB FECDisplayCB
    )

```

**Parameters:**

- nPort* Playing port number
- nFishPort* Fisheye port number
- FECDisplayCB* Fisheye display callback function, see details below:  

```

public void onFECDisplay
(
    int nPort,
    int nFishPort
);

```

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- Before you can call this API to set fisheye display callback function for fisheye dewarping, the live view should be started and the fisheye function should be enabled.
- This API must be called after calling [setDisplayCBEx](#). If `setDisplayCBEx` is not called, no information will be called back to Java layer.
- When positioning, the fisheye callback will be disabled, so the upper-layer should set again.

## 5.14.22 Set Absolute Rotation Parameters of 3D Fisheye

### Dewarping **setFish3DRotateAbs**

**API Definition:**

```

public boolean setFish3DRotateAbs(
    int nPort,
    int nSubPort,
    SRTRANS\_PARAM srTransParam
)

```

**Parameters:**

- nPort* Playing port number
- nSubPort* Set fisheye port No.
- srTransParam* Rotation configuration structure of 3D fisheye, see details in [SRTRANS\\_PARAM](#).

**Return Values:** Return *true* for success, and return *false* for failure.

## 5.14.23 Get Absolute Rotation Parameters of 3D Fisheye

### Dewarping **getFish3DRotate**

**API Definition:**

```

public boolean getFish3DRotate(
    int nPort,

```

```

        int          nSubPort,
        SRTRANS\_PARAM    srTransParam
    )

```

**Parameters:** *nPort* Playing port number  
*nSubPort* Set fisheye port No.  
*srTransParam* Rotation configuration structure of 3D fisheye, see details in [SRTRANS\\_PARAM](#).

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** This API should be called after calling [setWnd](#), otherwise, the obtained parameters are invalid.

### 5.14.24 Get Optimal Fisheye View Parameters (Wall Mounting) **getFish3DRotateSpecialView**

**API Definition:** `public boolean getFish3DRotateSpecialView(`  

```

        int          nPort,
        int          nSubPort,
        int          nSpecialViewType,
        SRTRANS\_PARAM    srTransParam
    )

```

**Parameters:** *nPort* Playing port number  
*nSubPort* Set fisheye port No.  
*nSpecialViewType* Set to 1, 0 is not supported.  
*srTransParam* Rotation configuration structure of 3D fisheye, see details in [SRTRANS\\_PARAM](#).

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** This API should be called after calling [setWnd](#) to set the display window, and it is for wall mounting only.

### 5.14.25 Set Animation for 3D Fisheye **setAnimation**

**API Definition:** `public boolean setAnimation(`  

```

        int          nPort,
        int          nSubPort,
        int          nAnimationType,
        int          nCurFrame,
        int          nTotalFrames
    )

```

**Parameters:** *nPort* Playing port number  
*nSubPort* Set fisheye port No.  
*nAnimationType* Set to 1, 0 is not supported.  
*nCurFrame* Current frame.

*nTotalFrames*      Total number of frames

**Return Values:** Return *true* for success, and return *false* for failure.

## 5.14.26 Set Fisheye Dewarping Parameters for EZVIZ Camera

### SetLDCFlag

**API Definition:**    public boolean    setLDCFlag(  
                      int     *nPort*,  
                      int     *nFlag*  
                      );

**Parameters:**    *nPort*                    Playing port number  
                      *nFlag*                Enable or disable fisheye dewarping of EZVIZ camera.

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- This API is customized for EZVIZ cameras, and it is only available for fisheye stream.
- This API does not supports hardware decoding. If it is called during hardware decoding, error code “16” will be returned.

## 5.15 Private Data

### 5.15.1 Display Private Data **renderPrivateData**

**API Definition:** `boolean renderPrivateData(`

```
    int    nPort,  
    int    nIntelType,  
    int    bTrue  
);
```

**Parameters:**

|                   |                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------|
| <i>nPort</i>      | Playing port number                                                                                 |
| <i>nIntelType</i> | Private data type, which is represented by bit, see details in the <a href="#">PRIVATE_RENDER</a> . |
| <i>bTrue</i>      | Whether to enable private information overlay.                                                      |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- This API is only available for stream with smart information.
- By default, the overlay of VCA information, motion detection information, and thermal information is enabled, the temperature information is disabled.
- During reversely playing, picture overlay is not supported.
- The actual overlay position of POS information is different with that of PC platform, and it varies with the display window size during digital zoom.

### 5.15.2 Display Private Data **renderPrivateDataEx**

**API Definition:** `boolean renderPrivateDataEx(`

```
    int    nPort,  
    int    nIntelType,  
    int    nSubType,  
    int    bTrue  
);
```

**Parameters:**

|                   |                                                                                                                                                                                                                                                    |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>nPort</i>      | Playing port number                                                                                                                                                                                                                                |
| <i>nIntelType</i> | Private data type, which is represented by bit, see details in the <a href="#">PRIVATE_RENDER</a> .                                                                                                                                                |
| <i>nSubType</i>   | Sub type of private data. When the <i>nIntelType</i> is "RENDER_FIRE_DET CET", the sub types are enumerated in <a href="#">FIRE_ALARM</a> ; when the <i>nIntelType</i> is "RENDER_TEM", the sub types are enumerated in <a href="#">TEM_FLAG</a> . |
| <i>bTrue</i>      | Whether to enable private information overlay.                                                                                                                                                                                                     |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- This API is only available for stream with smart information.
- By default, the overlay of VCA information, motion detection information, and

- thermal information is enabled, the temperature information is disabled.
- The overlay of POS information and picture is not supported.
- For thermal devcies, if the digital zoom is enabled, the private data overlay is not available.
- The characters in the thermal information or temperature is not supported to overlay.

### 5.15.3Set Private Data Callback Function

#### setAdditionalCallback

**Function:** boolean setAdditionalCallback(int nPort, int nSyncType, PlayerAdditionalCB additionCB)

**Parameters:**

|                    |            |                       |
|--------------------|------------|-----------------------|
| int                | nPort      | Playing channel No.   |
| int                | nSyncType  | Synchronization type  |
| PlayerAdditionalCB | additionCB | Private data callback |

PlayerAdditionalCB Callback

Function:

```
public void onAdditional (int nPort, int  
nType, int nStrVersion, int nTimeStamp,  
int nLength, byte[] data  
)
```

Function Parameter Description:

|        |             |                                                      |
|--------|-------------|------------------------------------------------------|
| int    | nPort       | Playing channel No.                                  |
| int    | nType       | Private data type                                    |
| int    | nStrVersion | Structure version returned by data for compatibility |
| int    | nTimeStamp  | Private data time stamp                              |
| int    | nLength     | Private data length                                  |
| byte[] | data        | Private data pointer                                 |

**Returned Value:** Return *true* on success, and return *false* on failure.

**Remarks:**

- This API is applicable for stream with VCA information. If there is no VCA information, this callback function will not response (mobile information and so on).
- The *nSyncType* is customized, the limitation will not be configured.

## 5.15.4 Set Private Data Callback Function

### setIVSDrawFunCallback

*boolean setIVSDrawFunCallback(int nPort, PlayerIVSDrawFunCB drawFunCB)*

**Parameters:**

**nPort:**

[in] number of the playing channel

**drawFunCB:**

[PlayerIVSDrawFunCB] function of private-data callback

**callback Function:**

[PlayerIVSDrawFunCB]

**nPort:**

[in] number of the playing channel

**nWidth:**

[in] width of the image

**nHeight:**

[in] height of the image

**nStamp:**

[in]time stamp of the current video frame

**nType:**

[in] type of the current frame(only supports T\_YV12 at present)

**nFrameRate:**

[in] frame rate of the video

**nFrameNum:**

[in] frame number of the video

**nDataType:**

[in] additional information type (smart information, vehicular information) synchronizing with the code streams

**nDataLen:**

[in] length of data

**data:**

[byte] data pointer

**Returned Value:**

true - API calling succeeds;

false - API calling fails.

**Remarks:**

Requires to enable private-data display (enabled in default). Customer can realize private-data display on getting private information via this port.

Do not support hardware decoding.

**Note:**

Suitable for code stream with VCA information. (associates with dots, line and frame)

### 5.15.5 Set Font Library Path **setOverlayPriInfoFlag**

**Function:** boolean setOverlayPriInfoFlag(int nPort, int nIntelType, int bTrue, String filePath)

**Parameters:**

|     |            |                                                                                                                                 |
|-----|------------|---------------------------------------------------------------------------------------------------------------------------------|
| int | nPort      | Playing channel No.                                                                                                             |
| int | nIntelType | Inner typr, and it is indicated by bit. Each bit refers to a type of private data. For details, refer to PLAYM4_PRIDATA_RENDER. |

PLAYM4\_PRIDATA\_RENDER Type

Description:

```
public static class PRIVATE_RENDER
{
    public static final int RENDER_ANA_INTEL_DATA    VCA
    = 1;
    public static final int RENDER_MD    Motion Detction
    = 2;
    public static final int RENDER_ADD_POS    POS Information Overlay
    = 4;
    public static final int RENDER_ADD_PIC    Picture Overlay
    = 8;
    public static final int RENDER_FIRE_DETCECT    Thermal Imaging Information
    = 16;
    public static final int RENDER_TEM    Temperature Information
    = 32;
}
```

|     |       |                                           |
|-----|-------|-------------------------------------------|
| int | bTrue | Display or not. 1-Display, 0-Not display. |
|-----|-------|-------------------------------------------|

|        |          |                      |
|--------|----------|----------------------|
| String | filePath | Picture saving path. |
|--------|----------|----------------------|

**Returned Value:** Return *true* on success, and return *false* on failure.

**Remarks:** Before calling this API, the Private Data Overlay should be enabled (it is enabled by default). The client will get the private information from this API and display the information.

**Notes:** This API is applicable for stream with VCA information (point, line, frame related).

## 5.15.6 Set the Background Frame Color of the Private Data with POS Information **setPosBGRectColor**

**API Definition:** `boolean setPosBGRectColor(`

```
    int  nPort,  
    int  nAlpha,  
    int  nRed,  
    int  nGreen,  
    int  nBlue
```

```
)
```

**Parameters:**

|               |                                                                       |
|---------------|-----------------------------------------------------------------------|
| <i>nPort</i>  | Playing port number.                                                  |
| <i>nAlpha</i> | Alpha color component (transparency), the value is between 0 and 100. |
| <i>nRed</i>   | Red color component, the value is between 0 and 255.                  |
| <i>nGreen</i> | Green color component, the value is between 0 and 255.                |
| <i>nBlue</i>  | Blue color component, the value is between 0 and 255.                 |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** For the private data without related information of background frame, parameters set by this API will not take effect.

## 5.16 Reverse Playing

### 5.16.1 Play Reversely ReversePlay

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function:</b>      | boolean ReversePlay(int nPort)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Parameters:</b>    | int nport      Playing channel No.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Return Values:</b> | Return 1 on success, and return 0 on failure.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Remarks:</b>       | <ul style="list-style-type: none"><li>● No audio played during reverse play. When the resolution is larger than or equal to 1080P, only 25 frames can be stored in the GOP. So when the interval between I frames is larger than 25 frames, the image will be unsmooth.</li><li>● You should create file index when playing reversely in the file mode (PlayM4_OpenFile). The supported maximum speed of the reverse play speed is slower than that of the play. TS packaging file is not supported.</li><li>● When in stream playing mode (PlayM4_OpenStream), reverse play is not supported by real-time data. For history stream data, i.e., recording files, they will be read and transmitted to playing reversely. The decoded data, which is transmitted in reverse order from application layer, should be a GOP. That is, parse the recording file to GOP, and then transmit the decoded data in reverse order according to the GOP timing sequence. For example, the user can read the recording files in local PC and parse them to GOPs (the frame decoded by I frame), the file time sequence is I1PPPPPPP..., I2PPPPPPP..., I3PPPPPPP...,....., InPPPPPPP..., while the sequence for transmitting is InPPPPPP..., In-1PPPPPP...,....., I3PPPPPPP..., I2PPPPPPP..., I1PPPPPPP...</li><li>● For SVAC stream, struck may happen when playing reversely.</li></ul> |

## 5.17 Others

### 5.17.1 Set Squeak Configurations **setHSPParam**

|                        |                                                                                                                                                                                                                                                                                                                         |              |                        |                |                |               |                                                               |              |                                          |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|------------------------|----------------|----------------|---------------|---------------------------------------------------------------|--------------|------------------------------------------|
| <b>API Definition:</b> | boolean setHSPParam(<br>int nPort,<br>int nEnable,<br>int nNotch,<br>int nTime<br>)                                                                                                                                                                                                                                     |              |                        |                |                |               |                                                               |              |                                          |
| <b>Parameters:</b>     | <table><tr><td><i>nPort</i></td><td>Playing channel number</td></tr><tr><td><i>nEnable</i></td><td>Enable/disable</td></tr><tr><td><i>nNotch</i></td><td>Set the filtering level (from 0 to 6), the default level is 4</td></tr><tr><td><i>nTime</i></td><td>Set squeak duration, the default is 500.</td></tr></table> | <i>nPort</i> | Playing channel number | <i>nEnable</i> | Enable/disable | <i>nNotch</i> | Set the filtering level (from 0 to 6), the default level is 4 | <i>nTime</i> | Set squeak duration, the default is 500. |
| <i>nPort</i>           | Playing channel number                                                                                                                                                                                                                                                                                                  |              |                        |                |                |               |                                                               |              |                                          |
| <i>nEnable</i>         | Enable/disable                                                                                                                                                                                                                                                                                                          |              |                        |                |                |               |                                                               |              |                                          |
| <i>nNotch</i>          | Set the filtering level (from 0 to 6), the default level is 4                                                                                                                                                                                                                                                           |              |                        |                |                |               |                                                               |              |                                          |
| <i>nTime</i>           | Set squeak duration, the default is 500.                                                                                                                                                                                                                                                                                |              |                        |                |                |               |                                                               |              |                                          |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** This API is customized for EZVIZ client software, so for setting more parameters, you should log in to the EZVIZ client.

## 5.17.2 Set Callback Function of PCM Audio Data

### setAudioDataCallback

**API Definition:**

```
public boolean setAudioDataCallBack(
    int nPort,
    PlayerAudioDataCB audioDataCB
)
```

**Parameters:**

- nPort* Playing port number
- audioDataCB* PCM audio data callback function, see details below:

```
public void onAudioData(
    int nPort, //Playing port number
    byte[] audioData, //PCM audio data
    int nDataLen, //Data size
    int nSampleRate //Audio sampling rate
);
```

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** This API is customized for video conference.

# Chapter 6 Structure and Enumeration

## 6.1 SRTRANS\_PARAM

**Description:** Rotation configuration structure of 3D fisheye.

**Structure Definition:**

```
public static class SRTRANS_PARAM
{
    public float ptzPositionX;
    public float ptzPositionY;
    public SRTRANS\_ELEMENT srtransElement;
    public int nTransCount;
}
```

**Members**

- ptzPositionX* X-coordinate of PTZ view center
- ptzPositionY* Y-coordinate of PTZ view center
- srtransElement* 3D rotation parameter structure, see details in [SRTRANS\\_ELEMENT](#).
- nTransCount* Reserved.

Related API: [setFish3DRotateAbs](#)

## 6.2 SRTRANS\_ELEMENT

**Description:** Parameter structure for rotating in 3D

**Structure Definition:** public static class SRTRANS\_ELEMENT

```
{
    public float    fAxisX;
    public float    fAxisY;
    public float    fAxisZ;
    public float    fValue;
}
```

**Members**

|               |                                 |
|---------------|---------------------------------|
| <i>fAxisX</i> | Value of rotating around X-axis |
| <i>fAxisY</i> | Value of rotating around Y-axis |
| <i>fAxisZ</i> | Value of rotating around Z-axis |
| <i>fValue</i> | Zoom in (-) or out (+)          |

Related API: [setFish3DRotateAbs](#)

## 6.3 FISHEYE\_PARAM

**Description:** Fisheye dewarping parameter structure

**Structure Definition:** public static class FISHEYE\_PARAM

```
{
    public int        updateType;
    public int        placeAndCorrect;
    public float       zoom;
    public float       wideScanOffset;
    public PTZ\_PARAM ptzParam;
    public CYCLE\_PARAM cycleParam;
}
```

**Members**

|                        |                                                           |
|------------------------|-----------------------------------------------------------|
| <i>updateType</i>      | Update types                                              |
| <i>placeAndCorrect</i> | Mounting and dewarping types                              |
| <i>zoom</i>            | Zooming parameters, range: [0.00001f, 1.0f]               |
| <i>wideScanOffset</i>  | Offset of 180° and 360° panorama, range: [0.00001, 360.0] |
| <i>ptzParam</i>        | PTZ dewarping parameters                                  |
| <i>cycleParam</i>      | Circle center parameters of fisheye view.                 |

Related API: [setParamFEC](#), [getParamFEC](#)

**Remarks:** When setting fisheye parameters, the order to take effect is: PTZ and zooming parameters>only PTZ parameters>circle center parameters>offset parameters>only zooming parameters.  
E.g., when the PTZ and circle center parameters are configured, if the PTZ parameters are failed to take effect, both the two configurations are failed; if the PTZ parameters are succeeded to take effect, but the circle center parameters

are failed to take effect, only the PTZ configurations are succeeded, and error will be returned by the API.

## 6.4 PTZ\_PARAM

**Description:** PTZ dewarping parameter structure

**Structure Definition:** public static class PTZ\_PARAM

```
{  
    public float    ptzPositionX;  
    public float    ptzPositionY  
}
```

**Members**

|                     |                                 |
|---------------------|---------------------------------|
| <i>ptzPositionX</i> | X-coordinate of PTZ view center |
| <i>ptzPositionY</i> | X-coordinate of PTZ view center |

**Related API:** [setParamFEC](#), [getParamFEC](#)

## 6.5 CYCLE\_PARAM

**Description:** Structure about circle center parameters of fisheye view.

**Structure Definition:** public static class CYCLE\_PARAM

```
{  
    public float    radiusLeft;  
    public float    radiusRight;  
    public float    radiusTop;  
    public float    radiusBottom  
}
```

**Members**

|                     |                               |
|---------------------|-------------------------------|
| <i>radiusLeft</i>   | X-coordinate of circle left   |
| <i>radiusRight</i>  | X-coordinate of circle right  |
| <i>radiusTop</i>    | Y-coordinate of circle top    |
| <i>radiusBottom</i> | Y-coordinate of circle bottom |

**Related API:** [setParamFEC](#), [getParamFEC](#)

**Remarks:**

- When two fisheye ports are enabled, and the circle center parameters of one of the ports are configured, if you call [delPortFEC](#) to delete a fisheye port, the image of the other fisheye port and the original image will be changed.
- For fisheye view's circle center, the left and top limits are between -0.5 and 0.4, the right and bottom limits are between 0.6 and 1.5, the left+right limit and top+bottom limit are between 0.8 and 1.2.

## 6.6 PRIDATA\_RENDER

**Description:** Enumerate private data types.

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Enumeration:</b> | <pre> public static class PRIVATE_RENDER {     public static final int  RENDER_ANA_INTEL_DATA  =1;     public static final int  RENDER_MD              =2;     public static final int  RENDER_ADD_POS         =4;     public static final int  RENDER_ADD_PIC         =8;     public static final int  RENDER_FIRE_DETCTET    =16;     public static final int  RENDER_TEM              =32;     public static final int  RENDER_BG_FONT         =64; } </pre> |
| <b>Members</b>      | <pre> RENDER_ANA_INTEL_DAT  VCA A RENDER_MD              Motion detection information RENDER_ADD_POS         POS information RENDER_ADD_PIC         Picture information (only supported by JPEG picture                         of EZVIZ) RENDER_FIRE_DETCTET    Thermal information RENDER_TEM              Temperature information RENDER_BG_FONT         Text information (with background frame) </pre>                                                     |
| <b>Related API:</b> | <a href="#">renderPrivateData</a>                                                                                                                                                                                                                                                                                                                                                                                                                               |

## 6.7 FIRE\_ALARM

|                     |                                                                                                                                                                                                                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description:</b> | Enumerate the sub types when the private data is thermal information.                                                                                                                                                                                                                   |
| <b>Enumeration:</b> | <pre> public static class FIRE_ALARM {     public static final int  FIRE_FRAME_DIS          = 1;     public static final int  FIRE_MAX_TEMP           = 2;     public static final int  FIRE_MAX_TEMP_POSITION  = 4;     public static final int  FIRE_DISTANCE           = 8; } </pre> |
| <b>Members</b>      | <pre> FIRE_FRAME_DIS          Fire point frame FIRE_MAX_TEMP           Highest temperature FIRE_MAX_TEMP_POSITION  Position information of highest temperature FIRE_DISTANCE           Position distance of highest temperature </pre>                                                  |
| <b>Related API:</b> | <a href="#">renderPrivateDataEx</a>                                                                                                                                                                                                                                                     |

## 6.8 TEM\_FLAG

|                     |                                                                           |
|---------------------|---------------------------------------------------------------------------|
| <b>Description:</b> | Enumerate the sub types when the private data is temperature information. |
| <b>Enumeration:</b> | <pre> public static class TEM_FLAG </pre>                                 |

|                     |                                                                                                                                                                                                            |                               |  |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|--|
|                     | <pre>{<br/>    public static final int  TEM_REGION_BOX          = 1;<br/>    public static final int  TEM_REGION_LINE        = 2;<br/>    public static final int  TEM_REGION_POINT       = 4;<br/>}</pre> |                               |  |
| <b>Members</b>      | TEM_REGION_BOX                                                                                                                                                                                             | Temperature measurement frame |  |
|                     | TEM_REGION_LINE                                                                                                                                                                                            | Temperature measurement line  |  |
|                     | TEM_REGION_POINT                                                                                                                                                                                           | Temperature measurement point |  |
| <b>Related API:</b> | <a href="#">renderPrivateDataEx</a>                                                                                                                                                                        |                               |  |

## Chapter 7 FAQ

### 7.1 Stream Mode

#### 7.1.1 Why does the error occur when transferring parameters to *PlayM4\_OpenStream*?

**Answers:**

**Steps:**

1. Check if the parameters are correct. When calling the interface *PlayM4\_OpenStream*, correct file header needs to be input (The correct file header content and file header length is *NET\_DVR\_SYSHEAD* obtained from the real time stream callback function in the network SDK and the *PktSysHeader* type obtained from the encoding callback function in compression card )
2. When calling failed and the returned error code is *PLAYM4\_ALLOC\_MEMORY\_ERROR*, it means the memory is not enough. You can check the current PC memory usage.

After the flow data operation, you can call *PlayM4\_CloseStream* interface to close the flow recovery system resource. *PlayM4\_CloseStream* and *PlayM4\_OpenStream* need to appear in pairs.

#### 7.1.2 How to perform fast forward when playing in stream mode?

**Answers:**

Fast forward is supported by stream mode, but it depends on the stream data and the network. By default, if the video frame rate is 25 fps, the stream will be play in  $\times 1$  speed. If you want to play in  $\times 2$  speed, the video frame rate should be twice larger than 25 fps.

#### 7.1.3 Why does the first frame display slower than other frames in the live view?

**Answers:**

- Check if the first frame is I-frame, if not, it needs to wait until I-frame to decode normally. Therefore, it needs long time.
- Poor network environment may cause this problem when real time stream live view.

When enabling Hard-Decoding Preferred, the stream meets the requirements. But when decoding has errors, it needs to change Soft-Decoding which may also cause this problem.

## 7.2 Live View

### 7.2.1 How to solve the struck phenomenon during the live view?

**Answers:**

**Steps:**

1. Check if the data is fluent, if not, please check the network.
2. Reduce the live view channels properly when multiple channels HD live view is stuck.
3. Check the current resolution. If the buffer is too small, the SDK buffer with the input data from PlayM4\_InputData may be full. You need to repeatedly input the data that is not successfully input.

**Suggestions:**

- ◆ If the resolution is 4CIF, the PlayM4\_OpenStream buffer can be set to 1.5~2M;
- ◆ If the resolution is 720P, the buffer can be set to 3~4M;
- ◆ If the resolution is 1080P, the buffer can be set to 6M;

**Note:** 1M is equal 1\*1024\*1024.

4. Check PlayM4\_SetDisplayBuf, you can set 15 nodes.
5. Save the data and play it. If it is stuck, please check if some data is missing in network; If it is normal, please check if the input data is right.

### 7.2.2 Why is there no image displayed after starting live view?

**Answers:**

**Steps:**

1. Check if the correct data is input into the playing buffer and decodes by calling PlayM4\_Play.
2. Check if the callback function PlayM4\_SetDecCallBack is registered that doesn't show image when decoding.
3. Check if the snapshot is successful. If not, please refer to step 1, or refer to step 4.
4. If it is successful, please snapshot continuously. If the pictures change with the scene and OSD, the data is normal and check if the screen handle is correct.
5. Save the decoding data as the local file and play it. If it can be played, please refer to step 6, or refer to step 7.
6. If it can be played normally, please check if the input data of PlayM4\_InputData is correct.
7. If it can't be played normally, please check if the data stream is correct. If it is the standard stream, you can use the third party decoder such as Elecard or VLC to verify. If the decoding is successful, please send the test stream to us to analyze. If not, please check if there is network package loss.
8. If the real time stream is in RTP format, please provide network package files in the following format: *the length of 4 byte RTP package + RTP data package* and send them to us.

### 7.2.3 How to solve the delay problem of live view?

**Answers:**

1. For the general devices, the delay is 100~200ms and the delay time has a big relationship with the network fluency. If the delay is normal at full frame rate and abnormal at low frame rate, please send the feedback to us. The delay is the minimum after version 6.2.2.2.
2. If the delay appears when multi-channel live view, please check the current CPU usage. If the CPU usage is at limitation (85% and above), please reduce some channels' live view or improve the hardware performance.
3. If the customer is using client software, you can set it good fluency. If the customer use SDK to secondary develop, you can call interface `PlayM4_SetDisplayBuf` to set the frame number of decoding buffer.
4. Set the interface `PlayM4_SetStreamOpenMode` to `STREAM_REALTIME`.
5. Check the network is good and the delay time of sending and receiving network packets is small.

## 7.3 Encrypted Stream

### 7.3.1 Why can't I play the encrypted stream?

**Answers:**

**Steps:**

1. Check if the stream is the baseline stream. If it is the custom stream, you need to contact the branch company to get the encryption type and the order number.
2. Check if the key is correct and the key setting interface `PlayM4_SetSecretKey` is calling properly.
3. You can send the stream to us.

## 7.4 Capture

### 7.4.1 How to save the captured picture in the memory for following processing?

**Answer:**

After decoding and playing normally (`PlayM4_Play`), you can call APIs to save captured pictures (`PlayM4_GetBMP` and `PlayM4_GetJPEG` ).

## 7.5 Play Audio

### 7.5.1 Why is it stuck when playing audio, but fluent when playing video?

**Answer:**

This problem may exist in the video and audio stream with low frame rate, or you can check the uniformity of audio data.

### 7.5.2 Why is there no sound when playing file?

**Answers:**

1. Check if there is audio data in stream and the audio has sound.
2. The audio package related type fields don't meet the rule, such as payload error in RTP package, stream type error in PS package, etc.
3. The audio is not encrypted but is encrypted in the package layer. After SDK version 6.2, it begins to focus on the audio encryption filed. If the previous version can play the audio normally, but the later version doesn't work, this may be the device problem. You need to check with the device.

## 7.6 File Mode

### 7.6.1 Why does the fast forward occur when playing file?

**Answer:**

Check if the frame rate or the timestamp of the stream is normal. You can also check if there is frame extracting.

### 7.6.2 Failed to open files

**Answers:**

- Check if the file header is correctly saved when saving files. For the device network SDK, the stream type in the real time stream callback function is NET\_DVR\_SYSHEAD. For the compression card, the current type in the encoding callback function is PktSysHeader.
- Check if the size of the file is less than 4K or greater than 4G.
- Check if there is I-frame. Video decoding relies on I-frame.

### 7.6.3 How to locate in file mode?

**Answer:**

Check if the file index is created successfully. If it is successful, check if the locating parameters such as time or frame number are set correctly.

## 7.7 Decoding Callback

### 7.7.1 How to get the decoded data of multiple channels?

**Answers:**

**Steps:**

1. One channel data corresponds to one playing port (get from PlayM4\_GetPort).
2. Call PlayM4\_SetStreamOpenMode to set the correct play mode.
3. Input the file header of this channel data into PlayM4\_OpenStream.
4. Call the register decoding callback function PlayM4\_SetDecCallBack to get the decoded data.
5. Call PlayM4\_Play to start the decoding thread. If you don't need to display the image, you can set screen handle NULL.
6. Call PlayM4\_InputData to input the correct data for each channel. The SDK supports multiple concurrent decoding and the current version supports up to 500 channels.

### 7.7.2 How to avoid frame loss when calling decoding callback function in live view?

**Answers:**

1. Real time record files and check if the data stream has some problems.
2. If the data has no problem, please comment all the implements in the decoding callback functions and check if there is frame loss. Also, the operation is more time-consuming when the decoding functions call each other which may cause this problem.
3. Check if PlayM4\_InputData has failed during the actual calling. If it has failed, please try to input the data again. If the error frequency is high, please set the size of buffer in PlayM4\_OpenStream bigger.

### 7.7.3 Why does the video frame obtained from decoding callback function only display a half height?

**Answer:**

The resolution of original image is 2CIF which means 704\*288 for PAL standard and 704\*240 for NTSC

standard. But the actual display is 704\*576 for PAL standard and 704\*480 for NTSC standard. In the decoding callback function, it gets the actual resolution of stream which causes this problem.

### **7.7.4 What's the difference between the decoding callback function and display callback function?**

**Answer:**

For decoding callback function, the decoded data will be returned to user; but for display callback function, only return the decoded and displayed video data to user.

### **7.7.5 Why is the timestamp information repeated when continuously getting three audio data in the decoding callback function?**

**Answer:**

Update the SDK version to the latest one. If you want to accurate audio frame information, you'd better use the stream analysis SDK.

## **7.8 Synchronous Playback**

### **7.8.1 How to realize the synchronous playback of multiple files?**

**Answer:**

1. All the files should contain the same time stamp.
2. Before calling synchronous playback API, make sure each file has an index.
3. During synchronous playback, you can execute positioning via the API: `setPlayedTimeEx`.

## **7.9 Others**

### **7.9.1 Failed to get the live view**

**Answer:**

1. The Android PlayCtrl Library SDK only support real device, not support virtual device.
2. Devices with main brand are suggested.



See Far, Go Further